

TensorBinding.jl

Package Function Reference

June 13, 2026

Contents

1	Overview	4
2	Central Data Structure: TBHamiltonian	5
3	Hamiltonian Construction (TBSystem.jl)	7
3.1	Constructor	7
3.2	Cache and Truncation Helpers	8
3.3	Incremental Hamiltonian Modification	9
3.4	Interaction Storage	11
3.5	Geometry Helpers	11
4	Low-Level MPO Builders (Hamiltonian.jl, 2D_lattice.jl)	12
4.1	1D Kinetic Operators	12
4.2	2D Kinetic Operators	12
4.3	Diagonal Mask MPOs	13
4.4	QTCI-Based Builders	14
4.5	Preset Model Hamiltonians (direct MPO)	14
4.6	Spatial LDOS Plotting Helpers	16
5	Generic nth-Neighbor Hopping (nnneighbor_tk.jl)	17
5.1	Shell Detection	17
5.2	Hopping Accumulator	17
6	Bilayer and Multilayer Hamiltonians	19
6.1	Commensurate Stacking (bilayer_tk.jl)	19
6.2	Twisted Multilayer (twisted_tk.jl)	19
6.3	Layer-Core Prepend/Postpend Helpers	20
7	Spin and Nambu Extensions (Supercond_tk.jl)	20
7.1	Index Constructors	20
7.2	Unified Prepend/Postpend Operators (utils.jl)	20
7.3	Spin and Nambu Wrappers	20
7.4	Antisymmetric Pairing MPO Builders	21
7.5	Higher-Level Assemblers	21
8	Kernel Polynomial Method (KPM_tk.jl)	21
8.1	Spectral Bounds Estimation	22
8.2	Building the Chebyshev List	22
8.3	Spectral Observables from Stored Chebyshev List	23

8.4	High-Level LDOS Dispatch (TBHamiltonian interface)	23
8.5	Spatial Exciton LDOS	25
8.6	HODC Kernel	26
9	Density Matrix Purification (Purification_tk.jl)	26
9.1	Initial Guess	26
9.2	McWeeny Purification	26
9.3	SP2 Purification	27
9.4	Unified Density-Matrix Wrapper	27
9.5	Sign of an MPO	27
9.6	LDoS and DoS via Density-Matrix Derivative	27
10	Quantum Fourier Transform and Band Structure (QFT_tk.jl)	28
10.1	QFT Conjugation	28
10.2	Exciton Momentum-Space Spectra (MPS-KPM)	28
10.3	Online Band Structure: <code>get_bands</code>	29
10.4	High-Symmetry k-Path Utilities	30
10.5	Auxiliary Index Projection Helpers	31
11	Topological Invariants (Topology_tk.jl)	31
11.1	Projector Retrieval	31
11.2	Quenched Position Operators	32
11.3	1D Winding Number	32
11.4	2D Real-Space Chern Marker	33
11.5	Valley Operator and Valley Chern Number (Honeycomb)	33
11.6	Thouless Pump	34
12	DMRG (DMRG_tk.jl)	35
13	Time Evolution (Timeev_tk.jl)	35
13.1	Propagator MPO and Single-Step TDVP	35
13.2	Trajectory Evolution: MPS States	35
13.3	State Analysis Utilities	36
13.4	Density Matrix Dynamics (open / driven systems)	36
13.5	Observables Along Trajectories	36
13.6	Diagnostic Tools	37
14	Two-Particle Systems (twoparticle_tk.jl)	37
14.1	High-Level Constructors	37
14.2	Low-Level MPO Builders	38
15	Random Phase Approximation (RPA_tk.jl)	38
15.1	High-Level TBHamiltonian API	39
15.2	Magnon (Transverse Spin) Susceptibility	39
15.3	Wynn ε -Accelerated RPA (Neumann Series)	39
15.4	Haydock Recursion (Operator-Level Krylov)	40
15.5	Double Chebyshev (2D) Bubble Decomposition	41
15.6	Low-Level Bubble Pipeline	42
15.7	Tensor Utilities (used internally by RPA)	42
16	Self-Consistent Mean Field (SCF_tk.jl)	43
16.1	Generic Driver	43
16.2	Channel-Specific Drivers	44

16.3 Hartree/Fock Builder Factories	45
16.4 Unified Entry Points	46
16.5 Density and Pairing Profile Extraction	47
16.6 Néel/Trivial Initial-Guess Density Matrices	48
17 Utilities (utils.jl)	48
17.1 Binary Encoding and MPS/MPO Conversions	48
17.2 Profile Evaluation and Spatial Sampling	49
17.3 QTCI Compression and MPS/MPO Builders	50
17.4 Shift and Hadamard Operators	50
18 Custom Operator Extensions	51
19 Quick-Reference Workflows	51
19.1 KPM Density of States and Band Structure	51
19.2 Density Matrix Purification	52
19.3 Topological Invariants	52
19.4 Bilayer and Twisted Multilayer	52
19.5 Spinful Superconductor (BdG)	53
19.6 Band Structure with Sublattice Resolution	53
19.7 aux_side: postpending auxiliary DOF	53
20 Krylov Green's Function (krylov_tk.jl)	53
21 Flake Masking (Flake_tk.jl)	54
21.1 SDF Primitives	54
21.2 CSG Combinators	55
21.3 Mask Application	55
22 GPU Acceleration (GPU_tk.jl)	55
22.1 Spectral and Spatial Maps (DOS/LDOS/Bands)	56
22.2 Topology	58
22.3 SCF (Magnetic Hubbard)	58
22.4 Experimental: Conductivity (Not Working Yet)	60
23 T-Junction Geometries (TJunction_tk.jl)	60
23.1 TJunction Index and Geometry	61
23.2 Hamiltonian Construction	61
24 Non-Hermitian Extensions (NH_tk.jl)	62
24.1 Hermitization	62
24.2 Non-Hermitian Model Building	63
24.3 Non-Hermitian KPM Spectral Function	64
25 Quasi-Particle Interference (QPI_tk.jl)	66

Overview

TensorBinding.jl is a Julia package for constructing and manipulating tight-binding Hamiltonians as Matrix Product Operators (MPOs) in the *quantics binary* (QTT) representation. A system of $N = 2^L$ sites is encoded in L qubit sites: site index $n \in \{0, \dots, N-1\}$ is identified with the binary string $b_1 b_2 \dots b_L$. This representation makes the bond dimension of many physically relevant Hamiltonians small (typically ≤ 10), enabling compression via Quantics Tensor Cross Interpolation (QTCI).

Key capabilities:

- MPO construction for 1D and 2D lattices (square, triangular, triangular Bravais, honeycomb, kagomé, Lieb, dice) via shift operators and QTCI. Sublattice-aware models carry an explicit sublattice index and exact real-space positions.
- Generic n th-nearest-neighbor hopping via `add_hopping_2D!`: uniform, direction-dependent, site-dependent, or fully position+direction-dependent amplitude functions on any 2D geometry.
- Bilayer, multilayer, twisted, and Bernal-stacked Hamiltonians.
- Spin- $\frac{1}{2}$ and Nambu (BdG) extensions via a prepend/postpend-core pattern, with auxiliary DOF placeable at front (`:pre`) or back (`:post`).
- Kernel Polynomial Method (KPM) for density of states, density matrices, spectral functions, and Green's functions.
- Density-matrix purification (McWeeny, SP2) with `TBHamiltonian`-aware API.
- Quantum Fourier Transform conjugation for band structure $A(k, \omega)$, supporting spinful, BdG, and layered Hamiltonians via `aux_proj`.
- Topological invariants: real-space Chern marker and winding-number density via quenched or flat position operators, compatible with any lattice geometry.
- DMRG ground state and spectral DMRG.
- Real-time evolution: TDVP (fixed and time-dependent H), propagator MPO.
- Random Phase Approximation (RPA) for susceptibility.
- Self-consistent mean-field (Hubbard model).

Site encoding conventions:

- *Position qubits*: L sites, site i (0-indexed) $\leftrightarrow$ binary string $b_1 \dots b_L$, big-endian.
- *Extended systems*: auxiliary indices can be *prepended* (`:pre`, default) or *postpended* (`:post`). For a spinful bilayer BdG system with `:pre` the site chain is `[nambu_s, spin_s, layer_s, pos_qubits...]`.
- *2D lattices*: row-major flattening, linear index $i = i_x + i_y \cdot 2^{L_x}$, $i_x \in \{0, \dots, 2^{L_x} - 1\}$, $i_y \in \{0, \dots, 2^{L_y} - 1\}$.
- *Honeycomb (legacy "hex_2d")*: intra-row bonds pair even and odd i_x sites; two inter-row bond types controlled by sublattice masks. Requires $L = L_x + L_y$ total qubits.
- *Sublattice-explicit models* ("`honeycomb`", "`honeycomb_nnn`", "`kagome`", "`lieb`", "`dice`"): use a triangular Bravais basis of $2^{L_x} \times 2^{L_y}$ unit cells ($L = L_x + L_y$ qubits) plus one postpended sublattice index of dimension $n_{\text{sub}} \in \{2, 3\}$. Total physical sites = $n_{\text{sub}} \cdot 2^L$; H.N counts unit cells.

Central Data Structure: TBHamiltonian

Defined in `src/core/TBSystem.jl`.

Field	Type	Description
<code>L</code>	<code>Int</code>	Number of position qubits; $N = 2^L$ sites.
<code>N</code>	<code>Int</code>	Number of physical sites per spin/Nambu/layer sector.
<code>sites</code>	<code>Vector{<:Index}</code>	All ITensor site indices (includes auxiliary indices if present).
<code>mpo</code>	<code>MPO</code>	The accumulated Hamiltonian MPO.
<code>geometry</code>	<code>Function</code> or <code>Nothing</code>	Callable <code>i -> pos_vec</code> (1-indexed); returns real-space position of physical atom <code>i</code> .
<code>geometry_uc</code>	<code>Function</code> or <code>Nothing</code>	Callable <code>i -> pos_vec</code> returning the Bravais unit-cell position, <i>identical</i> for all sublattice atoms in the same cell. Set for sublattice-aware models (<code>"honeycomb"</code> , <code>"honeycomb_nnn"</code> , <code>"kagome"</code> , <code>"lieb"</code> , <code>"dice"</code>); <code>nothing</code> for single-atom geometries.
<code>scale</code>	<code>Float64</code>	Energy half-bandwidth; <code>H/scale</code> has spectrum $\subseteq [-1, 1]$. Set to 0 for lazy DMRG estimation.
<code>center</code>	<code>Float64</code>	Spectral center $(E_{\max} + E_{\min})/2$. 0 for particle-hole symmetric spectra.
<code>spin_s</code>	<code>Index</code> or <code>Nothing</code>	Spin- $\frac{1}{2}$ index (set by <code>add_spin!</code>).
<code>nambu_s</code>	<code>Index</code> or <code>Nothing</code>	Nambu index (set by <code>add_superconductivity!</code>).
<code>layer_s</code>	<code>Index</code> or <code>Nothing</code>	Layer index (set by bilayer/multilayer constructors).
<code>sublattice_s</code>	<code>Index</code> or <code>Nothing</code>	Sublattice index postpended by sublattice-aware constructors (<code>honeycomb</code> , <code>kagomé</code> , <code>Lieb</code> , <code>dice</code>).
<code>aux_side</code>	<code>Symbol</code>	<code>:pre</code> (aux at front, default) or <code>:post</code> (aux at back).
<code>_tn_cache</code>	<code>Vector{MPO}</code> or <code>Nothing</code>	Cached MPO Chebyshev list $T_0 \dots T_N$ (mode <code>:mpo</code>).
<code>_tn_mps_cache</code>	<code>Vector{MPS}</code> or <code>Nothing</code>	Cached MPS Chebyshev list (mode <code>:mps</code>); set by <code>KPM_Tn</code> with <code>mode=:mps</code> .
<code>_tn_Ncheb</code>	<code>Int</code>	Order N of cached Chebyshev list (shared by both caches).
<code>_density_cache</code>	<code>MPO</code> or <code>Nothing</code>	Cached density matrix from purification.
<code>interaction_mpo</code>	<code>MPO</code> or <code>Nothing</code>	Hartree/CDW interaction kernel set by <code>add_interaction!</code> ; used by <code>get_scf(H, channel)</code> .
<code>fock_mpo</code>	<code>MPO</code> or <code>Nothing</code>	Fock/exchange interaction kernel set by <code>add_interaction!(H, V; channel=:fock)</code> .
<code>Lx</code>	<code>Int</code> or <code>Nothing</code>	Number of x -direction qubits for 2D geometries ($L_y = L - L_x$); <code>nothing</code> for 1D/non-rectangular geometries.

Do not construct directly; use `get_Hamiltonian` or a layer/multilayer constructor. Caches are invalidated automatically by all `add_*` functions and `truncate!`. The `center` field is set together with `scale` during spectral-bounds estimation and is needed to correctly rescale energies: $\tilde{\omega} = (\omega - \text{center})/\text{scale}$.

Hamiltonian Construction (TBSystem.jl)

Constructor

Signature	Description
<code>get_Hamiltonian(geometry, params; L, scale=nothing, tol=1e-8, maxdim=15, ref_sites=nothing, kwargs...)</code>	Build a <code>TBHamiltonian</code> from a named model. <code>geometry</code> is a string key; <code>params</code> holds model parameters (see below). <code>ref_sites</code> (optional): if supplied, the constructed MPO's site indices are replaced in-place with <code>ref_sites</code> via <code>fix_sites</code> , so that multiple Hamiltonians built independently share identical <code>Index</code> objects. Returns a <code>TBHamiltonian</code> .

Supported geometry strings:

geometry	params	Notes
"chain_1d"	hopping t	1D NN chain, direct MPO (no QTCI).
"square_2d"	hopping t	2D square, $2^{L_x} \times 2^{L_y}$ sites; Lx/Ly kwargs.
"hex_2d"	hopping t	2D honeycomb.
"triangular_2d"	hopping t	2D triangular.
"ssh"	(t, d)	SSH chain: hopping $t \pm d$ on alternating bonds.
"ssh_sublattice"	(t, d)	SSH chain with an explicit 2-dim SSH sublattice index (H.sublattice_s, aux_side=:post) instead of alternating bond amplitudes.
"aah"	(V, ϕ, t)	Aubry–André–Harper quasicrystal.
"uniform"	t	Uniform 1D with optional on-site v .
"haldane"	(t_2, ϕ, M) NamedTuple	Haldane model; requires rs ($N \times 2$ position matrix).
"chernhex"	(t, t_2, ms)	Haldane-type Chern insulator on honeycomb.
"chern8"	(V, t)	8-fold Chern mosaic on square lattice.
"qc2dsquare"	t	Quasicrystal-modulated square lattice.
"honeycomb"	$(t,)$ NamedTuple	Honeycomb with explicit 2-atom sublattice index (postpended); requires Lx/Ly kwargs.
"honeycomb_nnn"	(t, t_2) NamedTuple	Honeycomb with NN hopping t and NNN same-sublattice hopping t_2 .
"kagome"	$(t,)$ NamedTuple	Kagomé lattice, 3-atom unit cell; Lx/Ly required.
"lieb"	$(t,)$ NamedTuple	Lieb lattice, 3-atom unit cell; flat band at $E = 0$.
"dice"	$(t,)$ NamedTuple	Dice (T3) lattice, 3-atom unit cell (hub A, rim B/C).
"triangular_bravais"	hopping t	Bravais triangular lattice: $\mathbf{a}_1 = (1, 0)$, $\mathbf{a}_2 = (\frac{1}{2}, \frac{\sqrt{3}}{2})$, three clean bond types; Lx/Ly required.
"custom"	$f(i, j)$	Arbitrary hopping matrix; scale required.

Sublattice-aware geometries ("honeycomb", "honeycomb_nnn", "kagome", "lieb", "dice") postpend a sublattice_s index of dimension n_{sub} (2 or 3) to the site chain. The physical site count becomes $n_{\text{sub}} \cdot 2^{L_x+L_y}$; H.geometry maps the enlarged 1-indexed site number to a real-space 2D position, while H.geometry_uc maps it to the shared Bravais unit-cell position (same value for all n_{sub} atoms in a cell). H.Lx is set to L_x for these geometries and for all 2D presets ("square_2d", "hex_2d", "triangular_2d", "triangular_bravais", "chern8", "chernhex", "qc2dsquare"), and is nothing for 1D/custom geometries; add_hopping! dispatches on H.Lx to redirect to add_hopping_2D!.

Cache and Truncation Helpers

Signature	Description
<code>_invalidate_cache!(H) → H</code>	Clear all cached results (<code>_tn_cache</code> , <code>_tn_mps_cache</code> , <code>_density_cache</code> , <code>scale/center</code>). Called automatically by all <code>add_*</code> functions.
<code>truncate!(H; cutoff=1e-10, maxdim=nothing) → H</code>	Truncate <code>H.mpo</code> in-place; invalidates all caches.

Incremental Hamiltonian Modification

All functions below mutate `H` in place and invalidate the Chebyshev cache. `add_hopping!` and `add_onsite!` must be called before `add_spin!` or `add_superconductivity!`.

The optional `position` keyword (default: inherits `H.aux_side`) controls whether auxiliary indices are prepended (`:pre`) or postpended (`:post`).

Signature	Description
<code>add_hopping!(H, f; nn=1, boundary=:open, bc=nothing, maxdim=15, tol=1e-8, type=ComplexF64, apply_kwargs=NamedTuple(), sublat=nothing, sublat_from=nothing, sublat_to=nothing)</code>	Add a hopping term to <code>H.mpo</code> . The mode of <code>f</code> is selected by type: <code>f::Number</code> gives a uniform <code>nn</code> -th-neighbour hopping (shell <code>nn</code>); <code>f(i)</code> (1-arg Function) gives a spatially varying amplitude per site, QTCI-compressed; <code>f(i,j)</code> (2-arg Function) gives a full $N \times N$ matrix via QTCI (<code>nn</code> ignored). <code>boundary</code> (or its alias <code>bc</code>) is <code>:open</code> or <code>:periodic</code> , controlling the shift operators used for <code>nn</code> -th-neighbour terms. <code>apply_kwargs</code> is forwarded to the underlying <code>apply</code> call (e.g. <code>(maxdim=50, cutoff=1e-10,)</code>). If <code>H.Lx</code> is set (2D Hamiltonian), dispatches to <code>add_hopping_2D!</code> instead (sublattice keywords are not supported in that case). Without sublattice keywords, errors if any auxiliary DOF index is already present. Invalidates all caches.
<code>add_hopping!(H, f; sublat=k, ...)</code>	<i>Intra-sublattice</i> : requires <code>H.sublattice_s</code> . Restricts the full kinetic term to sublattice k by tensoring with the projector $ k\rangle\langle k $. Works with all three <code>f</code> modes; must be called before <code>add_spin!</code> / <code>add_superconductivity!</code> . <i>e.g.</i> <code>add_hopping!(H, -0.1; sublat=1, nn=2)</code> (NNN within sublattice A).
<code>add_hopping!(H, f; sublat_from=k, sublat_to=l, nn=1, ...)</code>	<i>Inter-sublattice</i> : requires <code>H.sublattice_s</code> and a scalar <code>f</code> . Adds $f \cdot K_u^{nn} \otimes l\rangle\langle k + \bar{f} \cdot K_d^{nn} \otimes k\rangle\langle l $ (Hermitian). <code>nn=0</code> gives an intra-cell bond (no position shift) — useful for correcting individual bond amplitudes in an existing sublattice Hamiltonian. <i>e.g.</i> <code>add_hopping!(H, -0.3; sublat_from=1, sublat_to=2, nn=1)</code> ($A \leftrightarrow B$ NN inter-cell).

<code>add_onsite!(H, f; layer=nothing, sublat=nothing, Lx=nothing, tol=1e-8, maxdim=nothing)</code>	Add a diagonal on-site potential to H. $f::\text{Number}$ gives a uniform constant ($f \cdot \text{Id}$, no QTCI); $f(\mathbf{n})$ (0-indexed unit-cell index $n \in \{0, \dots, N-1\}$) gives a spatially varying 1D potential; $f(\mathbf{i}_x, \mathbf{i}_y)$ (0-indexed 2D coordinates, $i_x = n \bmod N_x$, $i_y = n \div N_x$) requires the <code>Lx</code> keyword so $N_x = 2^{L_x}$. <code>sublat=k</code> restricts the potential to sublattice k via $k\rangle\langle k$ (requires <code>H.sublattice_s</code>); e.g. a Semenoff mass on honeycomb: <code>add_onsite!(H, +M; sublat=1),</code> <code>add_onsite!(H, -M; sublat=2).</code> <code>layer=k</code> (or a list of layer indices) restricts to the selected layer(s) (requires <code>H.layer_s</code>); <code>layer=nothing</code> (default) applies to all layers.
<code>add_spin!(H; cutoff=1e-8, maxdim=200, position=:pre)</code> <code>add_zeeman!(H, h; direction=:z, tol=1e-8, maxdim=200, position)</code>	Prepend (or postpend) a spin-$\frac{1}{2}$ index: $\mathbf{I}_{\text{spin}} \otimes H$. No-op if already spinful. Add Zeeman coupling $h \cdot S_\alpha$. h is a Number or function $h(i)$. <code>direction</code> $\in \{:\mathbf{x}, :\mathbf{y}, :\mathbf{z}\}$. Calls <code>add_spin!</code> automatically. If BdG is already applied, wraps in τ_z.
<code>add_superconductivity!(H, Δ; type=:swave, tol=1e-8, maxdim=200, position)</code>	BdG extension: prepend (or postpend) a Nambu index, with $H_{\text{BdG}} = \tau_z \otimes H_{\text{kin}} + \tau_+ \otimes H_{\text{pair}} + \tau_- \otimes H_{\text{pair}}^\dagger$. <code>type=:pwave</code> (spinless only, Δ a Number): antisymmetric NN pairing $\Delta(K_u - K_d)$ (Kitaev chain). <code>type=:swave</code> (default): for spinful H, singlet pairing $(i\sigma_y)_{\text{spin}} \otimes \Delta(r)$ with Δ a Number or 1-arg function $\Delta(i)$; for spinless H, on-site pairing is forbidden by Fermi antisymmetry, so a Number Δ auto-redirects to <code>type=:pwave</code> (printing an informational message) and a Function Δ errors (use <code>type=:custom</code> instead). <code>type=:custom</code>: arbitrary pairing $\Delta(i, j)$ via TCI (<code>pairing2MPO</code>). Errors if BdG has already been applied.
<code>add_soc!(H, λ; type=:rashba, direction=:z, tol=1e-8, maxdim=200, position)</code>	Spin-orbit coupling. Calls <code>add_spin!</code> automatically if needed. <code>type=:rashba</code>: NN Rashba $\lambda(i\sigma_y \otimes K_u - i\sigma_y \otimes K_d)$ (scalar λ only); breaks SU(2) spin symmetry while preserving time-reversal. <code>type=:ising</code>: diagonal $\lambda(i) \cdot S_z$ (Kane–Mele-type), λ a Number or 1-arg function. <code>type=:custom</code>: position-space MPO built from λ (Number, 1-arg function $\lambda(i)$, or 2-arg function $\lambda(i, j)$ via TCI), tensored with the spin operator given by <code>direction</code> ($\in \{:\mathbf{x}, :\mathbf{y}, :\mathbf{z}\}$). For Hermiticity, $\lambda(i, j) = \lambda(j, i)$ (automatic for diagonal/real-symmetric λ).

Typical workflow:

```
H = get_Hamiltonian("square_2d", 1.0; L=8)
```

```

add_onsite!(H, _ -> -0.5)          # chemical potential
add_zeeman!(H, 0.1)                # Zeeman splits spin sectors
add_soc!(H, 0.05)                  # Rashba SOC
add_superconductivity!(H, 0.2)     # s-wave BdG
# H.sites = [nambu_s, spin_s, pos_qubits...] (default :pre stacking)

```

Interaction Storage

Signature	Description
<code>add_interaction!(H, V; channel=:hartree, type=Float64, tol=1e-8, kwargs...) → H</code>	Store a pre-built interaction MPO in H for later use with <code>get_scf(H, channel)</code>. V can be an MPO (stored directly), a <code>Number</code> (scalar coupling $V \cdot \text{Id}$), a 1-arg <code>Function</code> $V(i)$ (0-indexed on-site potential, stored as a diagonal MPO), or a 2-arg <code>Function</code> $V(i, j)$ (0-indexed interaction kernel, stored as a full 2D MPO via <code>get_mpo</code>). <code>channel=:hartree</code> (or <code>:default</code>) stores into <code>H.interaction_mpo</code> (used by the <code>:cdw</code> and <code>:magnetic</code> SCF channels); <code>channel=:fock</code> (or <code>:exchange</code>) stores into <code>H.fock_mpo</code>. Does not invalidate the Chebyshev/density caches.

Geometry Helpers

Signature	Description
<code>honeycomb_positions(L; Lx=L div 2) → Matrix{Float64}</code>	Return $N \times 2$ real-space positions for a honeycomb patch ($N = 2^L$).
<code>square_positions(L; Lx=L div 2) → Matrix{Float64}</code>	Return $N \times 2$ real-space positions for a square lattice patch.
<code>triangular_positions(L; Lx=L div 2) → Matrix{Float64}</code>	Return $N \times 2$ real-space positions for a triangular lattice patch. Odd rows offset by 0.5 in x ; $y = i_y \cdot \sqrt{3}/2$.
<code>triangular_bravais_positions(L; Lx=L div 2) → Matrix{Float64}</code>	Positions for Bravais triangular lattice: $\mathbf{r}(i_x, i_y) = i_x \mathbf{a}_1 + i_y \mathbf{a}_2$ with $\mathbf{a}_1 = (1, 0)$, $\mathbf{a}_2 = (\frac{1}{2}, \frac{\sqrt{3}}{2})$.
<code>honeycomb_sublattice_positions(Lx, Ly) → Matrix{Float64}</code>	$2N_{\text{uc}} \times 2$ positions for honeycomb with explicit sublattice: A at $(i_x + 0.5 i_y, i_y \sqrt{3}/2)$, B at $(\dots + 0.5, \dots + \sqrt{3}/6)$.
<code>kagome_positions(Lx, Ly) → Matrix{Float64}</code>	$3N_{\text{uc}} \times 2$ positions for the kagomé lattice (A, B, C sublattices).
<code>lieb_positions(Lx, Ly) → Matrix{Float64}</code>	$3N_{\text{uc}} \times 2$ positions for the Lieb lattice (A, B, C sublattices).
<code>dice_positions(Lx, Ly) → Matrix{Float64}</code>	$3N_{\text{uc}} \times 2$ positions for the dice (T3) lattice (hub A, rim B/C).
<code>lattice_positions(lattice, Lx, Ly; angle_deg=0.0) → Matrix{Float64}</code>	Positions for <code>:square</code> , <code>:triangular</code> , or <code>:honeycomb</code> , optionally rotated by <code>angle_deg</code> degrees about the centroid. (<code>twisted.tk.jl</code>)

<code>central_index(geom, N) → Int</code>	1-indexed site whose position is closest to the
<code>central_index(H) → Int</code>	geometric centroid. <code>geom</code> is a callable <code>i → pos_vec</code> . Errors if <code>H.geometry</code> is <code>nothing</code> .
<code>_pos_sites(H) → Vector{<:Index}</code>	The L position-qubit indices, obtained by filtering <code>H.sites</code> to remove <code>H.spin_s</code> , <code>H.nambu_s</code> , <code>H.layer_s</code> , and <code>H.sublattice_s</code> (whichever are set), regardless of whether they sit at the front or back of <code>H.sites</code> .

Low-Level MPO Builders (Hamiltonian.jl, 2D_lattice.jl)

These functions build MPOs directly on a provided `sites` vector.

1D Kinetic Operators

Signature	Description
<code>kinetic_1d_nn(L, sites; boundary=:open, bc=nothing) → MPO</code>	Uniform NN hopping $\sum_i (i+1\rangle\langle i + \text{h.c.})$ via the shared <code>shift_pair_mpos</code> shift/adjoint pair. <code>boundary=:open</code> (default) is the open-chain convention; <code>boundary=:periodic</code> (or <code>bc=:pbc</code>) adds the wrap-around bond (<code>cyclic=true</code>).
<code>kinetic_1d_nn_custom(L, sites, hopping; boundary=:open, bc=nothing) → MPO</code>	Spatially varying NN hopping; <code>hopping</code> is a diagonal MPO of amplitudes, combined via <code>shift_hopping_mpo</code> . Same boundary/bc convention as <code>kinetic_1d_nn</code> .
<code>kineticNNN(L, sites, hopping, nn; apply_kwargs=NamedTuple(), boundary=:open, bc=nothing) → MPO</code>	1D chain with n -th-nearest-neighbour hopping and a spatially varying amplitude encoded as the diagonal MPO <code>hopping</code> , via the shared <code>shift_hopping_mpo</code> helper $(\text{hopping} \cdot \text{shift}(nn) + \text{shift}(nn)^\dagger \cdot \overline{\text{hopping}})$. <code>nn</code> ≥ 1 ; <code>boundary=:open</code> (default) or <code>:periodic/bc=:pbc</code> for a cyclic shift.
<code>generate_kin_u(sites, N) → MPO</code>	Shift MPO $ n+1\rangle\langle n $ (binary increment / up-shift).
<code>generate_kin_d(sites, N) → MPO</code>	Shift MPO $ n-1\rangle\langle n $ (binary decrement / down-shift).

2D Kinetic Operators

Signature	Description
<code>intrachain_hopping(L_chain, N, sites; hopping=Id, t=1) → MPO</code>	NN x -direction hopping on a 2D lattice with row-break mask applied.
<code>interchain_hopping_square(L_chain, N, sites; hopping=Id, t=1) → MPO</code>	NN y -direction hopping on a square lattice.
<code>interchain_hopping_square_2nd_plus(L_chain, N, sites; hopping=Id, t2=1) → MPO</code>	NN hopping along $(+x, +y)$ diagonal (shift $+L_{\text{chain}} + 1$); row-break mask prevents wrap-around.
<code>interchain_hopping_square_2nd_minus(L_chain, N, sites; hopping=Id, t2=1) → MPO</code>	NN hopping along $(+x, -y)$ diagonal (shift $-(L_{\text{chain}} - 1)$).

<code>interchain_hopping_triangle(L_chain, N, sites) → MPO</code>	Both diagonal inter-row bonds on a triangular lattice (SW↗NE and SE↖NW).
<code>interchain_hopping_honeycomb(L_chain, N, sites) → MPO</code>	Inter-row hopping for a honeycomb lattice. Two inequivalent bonds, each with forward and backward terms: upper bond (shift $+L_{\text{chain}} + 1$) masked by <code>odd_template × odd_skeleton</code> (odd- i_x columns, even rows); lower bond (shift $+L_{\text{chain}} - 1$) masked by <code>even_template × even_skeleton</code> (odd- i_x columns excluding $i_x=1$, odd rows).
<code>kineticintra2DNNN(Lx, Ly, sites, hopping, nn; apply_kwargs) → MPO</code>	Long-range intra-row hopping on $2^{L_x} \times 2^{L_y}$ square lattice. Row-boundary wrap suppressed by <code>_row_break_mpo</code> .
<code>kineticintra2DNNhex(Lx, Ly, sites, hopping, nn; apply_kwargs) → MPO</code>	Intra-row hopping for honeycomb; applies checkerboard A/B mask.
<code>kineticinterNNNSWNE(Lx, Ly, sites, hopping, nn; apply_kwargs) → MPO</code>	SW↗NE diagonal inter-row hopping (square lattice, <code>:xplus</code> mask).
<code>kineticinterNNNSENW(Lx, Ly, sites, hopping, nn; apply_kwargs) → MPO</code>	SE↖NW diagonal inter-row hopping (<code>:xplain</code> mask).
<code>kineticinterNNNtriSWNE(Lx, Ly, sites, hopping, nn; apply_kwargs) → MPO</code>	SW↗NE diagonal for triangular lattice; adds even-row selection mask.
<code>kineticinterNNNtriSEnw(Lx, Ly, sites, hopping, nn; apply_kwargs) → MPO</code>	SE↖NW diagonal for triangular lattice; adds odd-row selection mask.
<code>kineticinterNNNtri.bravais_diag(Lx, Ly, sites, hopping; apply_kwargs) → MPO</code>	Third-bond hopping for Bravais triangular lattice ($\Delta i_x = +1$, $\Delta i_y = -1$), linear shift $-(N_x - 1)$; row x -wrap at $i_x = N_x - 1$ suppressed by <code>_row_break_mpo(:xplus)</code> .

Diagonal Mask MPOs

Used internally as connectivity filters; all return diagonal 0/1 MPOs.

Signature	Description
<code>_row_break_mpo(Lx, Ly, sites; which::Symbol) → MPO</code>	Mask 0 at row boundaries. <code>which=:xplus</code> : last column; <code>:xplain</code> : first column.
<code>_row_select_mpo(_, Ly, sites; keep=:even) → MPO</code>	Retain only even or odd rows (<code>keep=:even</code> or <code>:odd</code> , 0-based).
<code>_col_select_mpo(Lx, Ly, sites; keep=:even) → MPO</code>	Retain only even or odd columns.
<code>_row_checker_mpo(Lx, Ly, sites) → MPO</code>	Checkerboard: 1 where $(i_x + i_y)$ is even, 0 otherwise.
<code>skeleton(L_chain, N, sites) → MPO</code>	0 where $i_x = 1$ (second column, 0-indexed), 1 elsewhere. Used internally for triangular wrap-around suppression.
<code>odd_template(_, N, sites) → MPO</code>	1 where i_x is odd (LSB of $i_x = 1$). Selects the A-sublattice columns for the honeycomb inter-row bonds.

<code>even_template(L_chain, N, sites) → MPO</code>	1 where i_x is odd and $i_x \neq 1$, i.e. $\text{proj}_{\text{odd } i_x} - \text{proj}_{i_x=1}$. Selects the B-sublattice columns, excluding the boundary column that wraps into the next row.
<code>odd_skeleton(L_chain, N, sites) → MPO</code>	1 where i_y is even (0-based, LSB of $i_y = 0$). Selects even rows for the upper honeycomb inter-row bond.
<code>even_skeleton(L_chain, N, sites) → MPO</code>	1 where i_y is odd (0-based, LSB of $i_y = 1$). Selects odd rows for the lower honeycomb inter-row bond.

QTCI-Based Builders

Signature	Description
<code>hopping2MPO(f, N, sites; tol=1e-8, initial_positions=[], type=Float64, unfoldingscheme=:interleaved) → MPO</code>	Compress $N \times N$ matrix $H[i, j] = f(i, j)$ into an MPO via QTCI. <code>unfoldingscheme=:interleaved</code> (default) or <code>:fused</code> .
<code>qtc_matrix_to_MPO(A_fun, L, sites; tol=1e-8, type=Float64) → MPO</code>	Like <code>hopping2MPO</code> for $2^L \times 2^L$ matrix functions; applies extra truncation at <code>maxdim=20</code> .
<code>get_diagonal_mpo(L, sites, f; type=Float64) → MPO</code>	Diagonal MPO with entry $f(x)$ at site $x \in \{1, \dots, 2^L\}$ (1-indexed). Compressed via 1D QTCI.
<code>qtt_mpo(L, xvals, sites, func; tol_quantics=1e-8, maxbonddim_quantics=50) → MPO</code>	Compress scalar function <code>func(x)</code> on grid <code>xvals</code> (e.g. <code>0:2^L-1</code>) into a diagonal L -site MPO via QTCI. Used throughout the preset Hamiltonians below to encode on-site potentials and hopping amplitudes; see also <code>get_diagonal_mpo</code> .
<code>compose_power(base, nn; side=:right, apply_kwargs) → MPO</code>	base^{nn} by exponentiation-by-squaring in $O(\log n)$ applies. Returns identity if <code>nn=0</code> .

Preset Model Hamiltonians (direct MPO)

Function	Description
<code>HUniform(L, t; v=1e-6, tol_quantics=1e-8, maxbonddim_quantics=10, nn=1)</code>	Uniform 1D chain with optional on-site potential <code>v</code> . <code>v=0</code> triggers a warning (constant functions can fail QTCI).
<code>HSSH(L, t, d; tol_quantics=1e-8, maxbonddim_quantics=10, nn=1)</code>	SSH dimerized chain: hopping $t \pm d$ on alternating bonds.
<code>HAAH(L, V, phi, t; b=(1+√5)/2, tol_quantics=1e-8, maxbonddim_quantics=50)</code>	Aubry–André–Harper: $H = -t \sum c_{i+1}^\dagger c_i + V \cos(2\pi b i + \phi) n_i$.

HUniform2DSquare(Lx, Ly, t; tol_quantics=1e-8, maxbondddim_quantics=10, cutoff=1e-10)	Uniform square lattice tight-binding: <code>kineticintra2DNNN</code> ($nn=1$) + <code>kineticNNN</code> ($nn=N_x$).
HUniform2Dhex(Lx, Ly, t; tol_quantics=1e-8, maxbondddim_quantics=10, cutoff=1e-10)	Uniform honeycomb lattice: <code>kineticintra2DNNhex</code> (checkerboard intra-row mask) + <code>kineticNNN</code> ($nn=N_x$).
HUniform2Dtri(Lx, Ly, t; tol_quantics=1e-8, maxbondddim_quantics=10, cutoff=1e-10)	Uniform triangular lattice (three kinetic terms: intra-row NN, y -hop, and both triangular diagonals via <code>kineticinterNNNtriSWNE</code> / <code>kineticinterNNNtriSEnw</code>).
HUniform2Dtri.bravais(Lx, Ly, t; tol_quantics=1e-8, maxbondddim_quantics=10, cutoff=1e-10)	Bravais triangular lattice: x , y , and diagonal bonds matching $\mathbf{a}_1 = (1, 0)$, $\mathbf{a}_2 = (\frac{1}{2}, \frac{\sqrt{3}}{2})$.
HChern8(Lx, Ly, V, t; a=5/64*2^Lx, t2=0.2t, tol_quantics=1e-8, maxbondddim_quantics=10, cutoff=1e-10)	8-fold Chern mosaic: alternating intra-row hopping plus complex NNN diagonal terms modulated by a 4-rotated-wavevector 8-fold pattern at amplitude V .
H2DChernhex(Lx, Ly, t, t2, ms; uniformhaldane=false, uniformsemenoff=false, tol_quantics=1e-8, maxbondddim_quantics=10, cutoff=1e-10)	Haldane Chern insulator on honeycomb; domain walls in t_2 and Semenoff mass at $x=N_x/2$ unless <code>uniformhaldane</code> / <code>uniformsemenoff</code> are set.
HQC2DSquare(Lx, Ly, t=1.0; tol_quantics=1e-8, maxbondddim_quantics=100, cutoff=1e-10)	Quasicrystal-modulated square lattice: bond-midpoint hopping amplitude from an 8-fold pattern with two competing wavevectors $b_1 = 5\sqrt{5}/2$, $b_2 = \sqrt{3}N_x/16$.
build_hamiltonian(model, L; mparams, mparam_dict) build_hamiltonian(model, Lx, Ly; mparams, mparam_dict)	Dispatch via <code>MODEL_REGISTRY</code> : build 1D MPO from model name and parameter string. 2D variant.

Sublattice-based builders (return `TBHamiltonian` directly):

Function	Description
<code>honeycomb_sublattice_hamiltonian(Lx, Ly, t=1; cutoff=1e-8, maxdim=200) → TBH</code>	Honeycomb NN Hamiltonian with 2-atom unit cell. 3 bond types: intra-cell A-B, x-intercell B→A, y-intercell B→A. <code>sublattice_s</code> dim-2 postpended; <code>scale=3.5 t </code> .
<code>honeycomb_nnn_hamiltonian(Lx, Ly, t=1, t2=0; cutoff=1e-8, maxdim=200) → TBH</code>	Honeycomb NN+NNN: adds same-sublattice NNN hopping t_2 (identity in sublattice space, $A \rightarrow A$ and $B \rightarrow B$) along the three triangular Bravais directions on top of NN t . <code>scale=3.5 t + 3.5 t2 </code> .

<code>kagome_hamiltonian(Lx, Ly, t=1; t_AB=t, t_AC=t, t_BC=t, cutoff=1e-8, maxdim=200) → TBH</code>	Kagomé lattice, 3-atom unit cell (A, B, C). Each of <code>t_AB</code> (intra-cell + x shift), <code>t_AC</code> (intra-cell + y shift), <code>t_BC</code> (intra-cell + diagonal shift N_x-1) defaults to <code>t</code> ; pass individually for anisotropic/breathing kagomé. Flat band at $E = -t$ (uniform case). <code>sublattice_s</code> dim-3 postpended; <code>scale</code> = $4.5 \max(t_{AB} , t_{AC} , t_{BC})$.
<code>lieb_hamiltonian(Lx, Ly, t=1; t_AB=t, t_AC=t, cutoff=1e-8, maxdim=200) → TBH</code>	Lieb lattice, 3-atom unit cell (A corner, B x -edge, C y -edge); no B-C bond. <code>t_AB</code> (A→B, intra-cell + x shift) and <code>t_AC</code> (A→C, intra-cell + y shift) default to <code>t</code> . Flat band at $E = 0$. <code>sublattice_s</code> dim-3 postpended; <code>scale</code> = $2.5 \max(t_{AB} , t_{AC})$.
<code>dice_hamiltonian(Lx, Ly, t=1; t_AB=t, t_AC=t, cutoff=1e-8, maxdim=200) → TBH</code>	Dice (T3) lattice, 3-atom unit cell: hub site A (coordination 6) couples to rim sites B and C. <code>t_AB</code> controls A-B bonds (intra-cell, x, y); <code>t_AC</code> controls A-C bonds (no intra-cell bond; x, y , and diagonal shift N_x+1). Doubly-degenerate flat band at $E = 0$. <code>sublattice_s</code> dim-3 postpended; <code>scale</code> = $4.5 \max(t_{AB} , t_{AC})$.
<code>ssh_sublattice_hamiltonian(L, t=1, d=0; cutoff=1e-8, maxdim=200) → TBH</code>	SSH chain with an explicit 2-component sublattice index (A=1, B=2, postpended). Intra-cell A→B amplitude $t + d$; inter-cell $B(n) \rightarrow A(n+1)$ amplitude $t - d$ via <code>generate_kin_u/generate_kin_d</code> . Periodic (the binary-increment wrap-around gives the $B(N-1) \rightarrow A(0)$ bond). Unlike the other sublattice builders this constructor also sets <code>geometry/geometry_uc</code> : atom i in unit cell $n = \lfloor (i-1)/2 \rfloor$ sits at $[n]$ (A) or $[n + 0.5]$ (B), with <code>geometry_uc(i) = [n]</code> for both. <code>scale</code> = $1.1(t + d + t - d)$.

Known models in MODEL_REGISTRY: "uniform", "ssh", "aah", "square_2d", "hex_2d", "triangular_2d", "triangular_bravais", "chern8", "chernhex", "qc2dsquare". Sublattice-aware geometries ("honeycomb", "honeycomb_nnn", "kagome", "lieb", "dice", "ssh_sublattice") are routed directly by `get_Hamiltonian` and are not in the plain MODEL_REGISTRY. `_parse_param_string(s) → Dict{Symbol,Any}` is the internal helper behind the `mparams` string: it splits "key1=val1, key2=val2" on commas/whitespace and auto-types each value as `Bool`, `Int`, `Float64`, or `String`.

Spatial LDOS Plotting Helpers

Scatter-plot helpers for the multi-atom sublattice geometries above, built on `kagome_positions/lieb_position` (see Geometry Helpers) and `get_ldos_spatial`.

Signature	Description
<code>plot_ldos_2d(ldos_mat, omegalist, omega_target; geometry=:honeycomb, Lx, Ly, markersize=12.0, colormap=:inferno, colorbar=true, clim=nothing, title="", kwargs...) → Plot</code>	Scatter plot of the spatial LDOS at the energy in <code>omegalist</code> nearest <code>omega_target</code> . <code>ldos_mat</code> is the (<code>N_omega</code> × <code>n_atoms</code>) matrix from <code>get_ldos_spatial</code> called with <code>num_x=H.N</code> ; column k maps to atom k of <code>geometry</code> 's position matrix. <code>geometry</code> ∈ { <code>:honeycomb</code> , <code>:kagome</code> , <code>:lieb</code> , <code>:dice</code> }. Errors if <code>size(ldos_mat, 2) ≠</code> the atom count for (Lx, Ly).

<code>plot_ldos_multilayer(ldos_layers,</code>	Top-view scatter plot of multilayer LDOS.
<code>omegalist, omega_target;</code>	<code>ldos_layers</code> is a <code>Vector</code> of per-layer
<code>stacking=:Bernal,</code>	$(N_{\text{omega}} \times n_{\text{atoms}})$ matrices (one per layer,
<code>geometry=:honeycomb, Lx, Ly,</code>	<code>proj_layer=k</code>). <code>stacking=:AA</code> : all layers share
<code>markersize=10.0,</code>	positions, LDOS summed directly.
<code>colormap=:inferno,</code>	<code>stacking=:Bernal</code> (default): odd layers sit at the
<code>colorbar=true, clim=nothing,</code>	standard geometry positions, even layers are
<code>title="", kwargs...) → Plot</code>	displaced by the intra-cell A→B bond vector; both
	groups are plotted together ($2 \times n_{\text{atoms}}$ points),
	each summed over its own (odd/even) layers.

Generic *n*th-Neighbor Hopping (nnneighbor_tk.jl)

These functions operate on any 2D TBHamiltonian regardless of geometry, adding *n*th-nearest-neighbor bonds detected automatically via a reference patch.

Shell Detection

Signature	Description
<code>get_shell_disps(H, nn; Lx, Ly) → Vector{NTuple{4,Int}}</code>	Print and return the canonical displacement tuples $(\Delta i_x, \Delta i_y, s_{\text{from}}, s_{\text{to}})$ for the <i>n</i> th-nearest-neighbour shell. “Canonical” means $\Delta i_x > 0$, or $(\Delta i_x = 0 \text{ and } \Delta i_y > 0)$, or $(\Delta i_x = \Delta i_y = 0 \text{ and } s_{\text{from}} < s_{\text{to}})$. Call this first to discover which tuples will be passed to an amplitude function.

Hopping Accumulator

Signature	Description
<code>add_hopping_2D!(H, f; Lx, Ly, nn=1, layer=nothing, lattice=nothing, geometry=nothing, maxdim=50, tol=1e-8) → H</code>	Add <i>n</i> th-nearest-neighbor hopping to H in place. Iterates over canonical shell displacements; for each, builds a forward+backward shift MPO weighted by the amplitude returned by <code>f</code> . Works for all 2D geometries including sublattice-aware ones (honeycomb, kagomé, Lieb, dice).

Layered Hamiltonians (`H.layer_s` set, requires `H.spin_s/H.nambu_s` both `nothing`): `layer=nothing` applies the hopping to every layer; an integer or collection of integers (1-indexed) restricts it to those layers. The shell term is built once on a scratch position(+sublattice) Hamiltonian and prepended per selected layer via `prepend_layer_projector`. If `H.geometry` is not already set, pass `lattice=:square/:triangular/:honeycomb` or an explicit `geometry` (a function $i \rightarrow r_i$, or an $N \times d$ position matrix with one row per position site, or per atom when `H.sublattice_s` is set).

Amplitude function forms accepted by `f`:

Signature of <code>f</code>	Meaning
<code>f::Number</code>	Uniform amplitude for all bonds in the shell.
<code>f($\Delta ix, \Delta iy, fs, ts$) -> Number</code>	Direction-dependent; receives canonical displacement and sublattice indices.
<code>f(n) -> Number</code>	Site-dependent (0-indexed unit-cell index n).
<code>f(ix, iy) -> Number</code>	Site-dependent via explicit 2D unit-cell coordinates.
<code>f($n, \Delta ix, \Delta iy, fs, ts$) -> Number</code>	Both site (0-indexed) and direction.
<code>f($ix, iy, \Delta ix, \Delta iy, fs, ts$) -> Number</code>	Full control: explicit 2D coords and direction.

Sublattice convention: For sublattice-aware geometries, `fs/ts` are 1-indexed sublattice numbers. The forward operator uses matrix element $M_{\text{fwd}}[s_{\text{from}}, s_{\text{to}}] = 1$, consistent with the original Hamiltonian builders.

Typical usage:

```
# Inspect which displacements will be passed (honeycomb, nn=2)
TensorBinding.get_shell_disps(H, 2; Lx=Lx, Ly=Ly)
# [(1,0,1,1), (0,1,1,1), (1,-1,1,1), (1,0,2,2), (0,1,2,2), (1,-1,2,2)]

# Uniform NNN hopping (same-sublattice, real amplitude)
add_hopping_2D!(H, 0.2; Lx=Lx, Ly=Ly, nn=2)

# Direction-dependent complex amplitude (Haldane-like phases)
add_hopping_2D!(H,
    (dx,dy,fs,ts) -> (dx,dy,fs,ts)==(1,0,1,1) ? t2*exp(im*phi) : 0.0im;
    Lx=Lx, Ly=Ly, nn=2)

# Position+direction dependent modulated hopping
add_hopping_2D!(H,
    (ix,iy,dx,dy,fs,ts) -> t1 + amp*sin(dot(G_vec[(dx,dy,fs,ts)], r_mid(ix,iy)));
    Lx=Lx, Ly=Ly, nn=1)
```

Bilayer and Multilayer Hamiltonians

Commensurate Stacking (bilayer.tk.jl)

Site encoding: site 1 = layer qubit (Index of dim n_{layers}); sites $2 \dots L + 1$ = position qubits. Constructors return a TBHamiltonian with H.layer_s set.

Signature	Description
<code>bilayer_hamiltonian(lattice, Lx, Ly; stacking=:AA, t_intra=1.0, t_inter=0.3, sublattice=false, cutoff=1e-8, maxdim=200) → TBHamiltonian</code>	Two-layer Hamiltonian: $\sum_k P_k \otimes H_{\text{mono}} + (1\rangle\langle 2 + 2\rangle\langle 1) \otimes V$. stacking=:AA (on-site) or :Bernal (A_1 – B_2 , honeycomb only). sublattice=true appends an explicit sublattice Index (H.sublattice_s, dim=2, currently :honeycomb only): the monolayer term comes from <code>honeycomb_sublattice_hamiltonian</code> and interlayer coupling is built per-sublattice (on-site identity for :AA , or $A_1 \rightarrow B_2/B_2 \rightarrow A_1$ for :Bernal), with H.aux_side=:pre .
<code>multilayer_hamiltonian(lattice, Lx, Ly, n_layers; stacking=:AA, t_intra=1.0, t_inter=0.3, sublattice=false, cutoff=1e-8, maxdim=200) → TBHamiltonian</code>	Generalises to n_{layers} layers; interlayer coupling between adjacent pairs only. sublattice=true behaves as for <code>bilayer_hamiltonian</code> , applied to every adjacent layer pair $(k, k+1)$.
<code>interlayer_mpo(lattice, stacking, Lx, Ly, sites; t_inter=1.0, cutoff=1e-8) → MPO</code>	Build the position-space interlayer coupling MPO V . Routes to <code>_bernal_interlayer_mpo</code> or <code>_aa_interlayer_mpo</code> .
<code>_bernal_interlayer_mpo(L, sites; t_inter=1.0, cutoff=1e-8) → MPO</code>	Exact Bernal A_1 – B_2 coupling: $V = t_{\perp}(K_u D_A + D_A K_d)$. D_A projects onto odd (A) sites (1-indexed); $K_{u/d}$ are ± 1 shifts.
<code>_aa_interlayer_mpo(sites; t_inter=1.0) → MPO</code>	AA stacking: on-site coupling $V = t_{\perp} \mathbf{I}$.

Twisted Multilayer (twisted.tk.jl)

Signature	Description
<code>twisted_multilayer_hamiltonian(lattice, Lx, Ly, angles_deg; t_intra=1.0, t_inter=0.3, $\alpha_{\text{decay}}=1/16$, tol=1e-6, cutoff=1e-8, maxdim=200) → TBHamiltonian</code>	Twisted multilayer: layer k rotated to angles_deg[k] . Interlayer coupling $V_{kl}(i, j) = t_{\perp} e^{-\alpha \mathbf{r}_k^{(i)} - \mathbf{r}_l^{(j)} }$ compressed via QTCI.
<code>twisted_bilayer_hamiltonian(lattice, Lx, Ly, θ_{deg}; ...) → TBHamiltonian</code>	Convenience wrapper: layer 1 at 0° , layer 2 at θ_{deg} .
<code>monolayer_hamiltonian(lattice, Lx, Ly, sites; t=1.0, cutoff=1e-8) → MPO</code>	Single-layer NN Hamiltonian MPO for :square , :triangular , or :honeycomb .

Layer-Core Prepend/Postpend Helpers

These functions extend an L -site MPO by one site by prepending or postpending a single-site layer operator. All are thin wrappers around `prepend_op/postpend_op`.

Signature	Description
<code>prepend_layer_projector(H, layer_s, k) → MPO</code>	Prepend $ k\rangle\langle k $ (1-indexed). Used for $P_k \otimes H$.
<code>prepend_layer_hopping(H, layer_s, k, l) → MPO</code>	Prepend $ k\rangle\langle l $. Used for interlayer $ k\rangle\langle l \otimes V$.
<code>postpend_layer_projector(H, layer_s, k) → MPO</code>	Postpend $ k\rangle\langle k $ to the end of H.
<code>postpend_layer_hopping(H, layer_s, k, l) → MPO</code>	Postpend $ k\rangle\langle l $ to the end of H.

Spin and Nambu Extensions (Supercond.tk.jl)

Index Constructors

Function	Description
<code>spin_index()</code>	Create a dim-2 Index tagged "Spin" (1= $\uparrow$, 2= $\downarrow$).
<code>nambu_index()</code>	Create a dim-2 Index tagged "Nambu" (1=particle, 2=hole).

Unified Prepend/Postpend Operators (utils.jl)

Signature	Description
<code>prepend_op(H, s, mat) → MPO</code>	General prepend: extends H by prepending an arbitrary matrix <code>mat</code> on site <code>s</code> .
<code>prepend_op(H, s, op::Symbol) → MPO</code>	Named operator lookup for "Spin" or "Nambu" tagged indices.
<code>prepend_op(H, s, k, l) → MPO</code>	Sparse $ k\rangle\langle l $ on site <code>s</code> (any dimension; 1-indexed).
<code>prepend_op(H, s, k) → MPO</code>	Projector $ k\rangle\langle k $.
<code>postpend_op(H, s, mat / op / k, l / k) → MPO</code>	Symmetric counterpart of <code>prepend_op</code> ; appends to the end of H.

Spin and Nambu Wrappers

Signature	Description
<code>prepend_spin(H, spin_s, op) → MPO</code>	Prepend a spin- $\frac{1}{2}$ operator. <code>op</code> is a <code>Symbol</code> or 2×2 matrix. Built-in ops: <code>:Id</code> , <code>:Pup</code> , <code>:Pdn</code> , <code>:Sz</code> , <code>:Sp</code> , <code>:Sm</code> , <code>:Sx</code> , <code>:Sy</code> , <code>:iSy</code> , <code>:miSy</code> .
<code>postpend_spin(H, spin_s, op) → MPO</code>	Postpend a spin- $\frac{1}{2}$ operator to the end of H. Same ops as above.
<code>prepend_nambu(H, nambu_s, op) → MPO</code>	Prepend a Nambu (particle-hole) operator. Built-in ops: <code>:Id</code> , <code>:Pp</code> , <code>:Ph</code> , <code>:tz</code> , <code>:tx</code> , <code>:ty</code> , <code>:tp</code> , <code>:tm</code> .

<code>postpend_nambu(H, nambu_s, op) → MPO</code>	Postpend a Nambu operator to the end of H. Same ops as above.
---	--

Antisymmetric Pairing MPO Builders

Pairing terms $\Delta(i, j)$ for `bdg_hamiltonian/bdg_spin_hamiltonian` must be antisymmetric, $\Delta(i, j) = -\Delta(j, i)$, since $c_i^\dagger c_j^\dagger = -c_j^\dagger c_i^\dagger$. These builders construct such Δ as an MPO `H_pair`.

Signature	Description
<code>pairingNNN(L, sites, hopping, nn; apply_kwargs=NamedTuple()) → MPO</code>	Exact n th-neighbor antisymmetric pairing on a 1D chain: $H_{\text{pair}} = \text{hopping} \cdot K_f^{nn} - K_b^{nn} \cdot \text{dag}(\text{hopping})$, where $K_{f/b}$ are the quantics forward/backward unit-shift MPOs (built via <code>compose_power</code>). Antisymmetric analogue of <code>kineticNNN</code> . For uniform pairing $\text{hopping} = \Delta \cdot I$ this gives $H_{\text{pair}} = \Delta \cdot (K_f^{nn} - K_b^{nn})$.
<code>pairing2MPO(f, N, sites; tol=1e-8, type=ComplexF64, initial_positions=[], unfoldingscheme=:interleaved) → MPO</code>	QTCI compression of a general pairing matrix $\Delta(i, j) = f(i, j)$; thin wrapper around <code>hopping2MPO</code> . The caller must supply an antisymmetric $\mathbf{f}$ ($f(i, j) = -f(j, i)$). Prefer <code>pairingNNN</code> for nearest-neighbor pairing (exact, no TCI); use <code>pairing2MPO</code> for longer-range or d -wave/ $p \pm ip$ patterns.

Higher-Level Assemblers

Signature	Description
<code>spin_hamiltonian(H_up, H_dn, spin_s; H_Zeeman=nothing, cutoff=1e-8) → MPO</code>	$P_\uparrow \otimes H_\uparrow + P_\downarrow \otimes H_\downarrow [+S_z \otimes H_{\text{Zeeman}}]$.
<code>bdg_hamiltonian(H_kin, H_pair, nambu_s; cutoff=1e-8) → MPO</code>	Spinless BdG: $\tau_z \otimes H_{\text{kin}} + \tau_+ \otimes H_{\text{pair}} + \tau_- \otimes H_{\text{pair}}^\dagger$.
<code>bdg_spin_hamiltonian(H_up, H_dn, H_pair, spin_s, nambu_s; H_soc=nothing, cutoff=1e-8) → MPO</code>	Spin- $\frac{1}{2}$ singlet BdG in Nambu-spin space. Pairing: $\tau_+ \otimes (i\sigma_y) \otimes H_{\text{pair}} + \text{h.c.}$. Optional Ising SOC $\tau_z \otimes S_z \otimes H_{\text{soc}}$.

Kernel Polynomial Method (KPM.tk.jl)

The KPM expands spectral quantities in Chebyshev polynomials $T_n(\tilde{H})$ where $\tilde{H} = (H - \text{center})/\text{scale} \in [-1, 1]$. Supported kernels: `:jackson` (default), `:lorentz`, `:fejer`, `:dirichlet`, `:hodc`.

Three complementary Chebyshev strategies are available, trading memory for generality:

- **MPO mode** (`mode=:mpo`): stores the full operator $T_n(\tilde{H})$ as an MPO. Needed for spectral functions at arbitrary sites, Green's functions, and density matrices. Bond dimension grows with n ; control with `maxdim`.
- **Diagonal/online mode**: extracts only the diagonal of each T_n as an MPS. Scales as $O(N_{\text{cheb}})$ MPO operations regardless of the number of energy points. Used by `get_ldos_spectrum`, `get_ldos_online`, and `get_ldos_spatial`.

- **MPS mode** (`mode=:mps`): propagates a reference state $|\phi_n\rangle = T_n(\tilde{H})|\psi_0\rangle$. Site-resolved LDoS at ψ_0 without storing any operator. Required for exciton systems (2L-site chain makes MPO mode prohibitive).

Spectral Bounds Estimation

Signature	Description
<code>_estimate_spectral_bounds(H_mpo, sites; dmrg_nsweeps=5, dmrg_maxdim=[10,20,40], dmrg_linkdim=4) → (scale, center)</code> <code>_ensure_scale!(H::TBHamiltonian; ...)</code>	Run DMRG to minimise H and $-H$, yielding $E_{\min}/E_{\max}$. Returns $\text{scale} = (E_{\max} - E_{\min})/2 \times 1.1$ and $\text{center} = (E_{\max} + E_{\min})/2$. If $H.\text{scale}==0$, call <code>_estimate_spectral_bounds</code> and store in $H.\text{scale}/H.\text{center}$. No-op otherwise. Called lazily by all high-level KPM functions.

Building the Chebyshev List

Signature	Description
<code>KPM_Tn(H_mpo, N, sites; scale=nothing, center=0.0, maxdim=40, cutoff=1e-8, ...) → (Tn_list, scale, center)</code>	Low-level MPO form. Build Chebyshev MPOs $T_n(\tilde{H})$ for $n = 0 \dots N$. If <code>scale=nothing</code> , bounds estimated via DMRG.
<code>KPM_Tn(H::TBHamiltonian, Ncheb; mode=:mpo, psi0=nothing, maxdim=40, cutoff=1e-8, ...) → (Tn_list, scale, center)</code>	High-level form. Calls <code>_ensure_scale!</code> lazily. <code>mode=:mpo</code> : caches MPO list in $H._\text{tn_cache}$. <code>mode=:mps</code> : requires <code>psi0</code> ; caches MPS list in $H._\text{tn_mps_cache}$. Both update $H._\text{tn_Ncheb}$.
<code>KPM_Tn(H::TBHamiltonian, Ncheb, X; maxdim=40, cutoff=1e-8, ...) → (Tn_mps_list, scale, center)</code>	Exciton convenience form. Builds MPS chain $T_n(\tilde{H}) X, X\rangle$ for 1-indexed exciton site X ; caches in $H._\text{tn_mps_cache}$. No MPO mode provided for 2L-site exciton systems.
<code>KPM_Tn_mps(H_mpo, N, psi0, sites; scale=nothing, center=0.0, maxdim=40, cutoff=1e-8, ...) → (Tn_mps_list, scale, center)</code>	Low-level MPS form. Builds $ \phi_n\rangle = T_n(\tilde{H}) \psi_0\rangle$ for $n = 0 \dots N$. Moments: $\mu_n = \langle \psi_0 \phi_n \rangle$.
<code>KPM_Tn_mps(H::TBHamiltonian, N, psi0; maxdim=40, cutoff=1e-8, ...) → (Tn_mps_list, scale, center)</code>	High-level MPS form. Calls <code>_ensure_scale!</code> ; caches in $H._\text{tn_mps_cache}$.

Important — return order: The return tuple is `(Tn_list, scale, center)`. A common bug is `Tn, _, scale = KPM_Tn(...)`, which assigns `center` to `scale`. The correct pattern is:

```
Tn, scale, center = KPM_Tn(H, Ncheb, sites)
omega_r = (omega - center) / scale    # rescaled energy in (-1, 1)
```

When using the `TBHamiltonian` overload, `H.scale` and `H.center` are updated automatically and can be read directly.

Spectral Observables from Stored Chebyshev List

Signature	Description
<code>get_ldos_w_from_Tn(Tn_list, N, ω; maxdim=40, kernel=:jackson, ...) $\rightarrow$ MPO</code>	Full spectral MPO $\delta(\omega - H)$ at rescaled $\omega \in (-1, 1)$. Legacy full-MPO path; prefer <code>get_ldos</code> with <code>mode=:diag</code> for site-resolved LDOS.
<code>get_ldos_diag_from_Tn(Tn_list, N, ω_vals; kernel=:jackson, lambda=4.0, maxdim=40, cutoff=1e-8) $\rightarrow$ Vector{Union{Nothing,MPS}}</code>	Diagonal-MPS LDOS at all energies in a single pass over <code>Tn_list</code> . Returns one MPS $A(r, \omega)$ per energy (nothing for $ \omega \geq 1$).
<code>get_Green_retarded_from_Tn(Tn_list, N, ω; η=1e-2, maxdim=40, kernel=:jackson, ...) $\rightarrow$ MPO</code>	Retarded Green's function MPO $G^R(\omega + i\eta) = (\omega + i\eta - H)^{-1}$.
<code>get_density_from_Tn(Tn_list, N; fermi=0.0, maxdim=40, kernel=:jackson, ...) $\rightarrow$ MPO</code>	Density matrix $\rho = \theta(\mu - H)$ at rescaled Fermi level <code>fermi</code> .
<code>get_ldos_from_mun(mun_list, N, E; kernel=:jackson, lambda=4.0, ...) $\rightarrow$ Real</code>	Reconstruct LDoS at rescaled energy E from moment list $\mu_n = \langle \psi_0 T_n(\tilde{H}) \psi_0 \rangle$ (MPS mode output).
<code>get_ldos_hodc_from_mun(mun_list, N, E; eta=0.02, m_order=4) $\rightarrow$ Real</code>	HODC (Higher-Order Delta Chebyshev) variant of <code>get_ldos_from_mun</code> ; contour-based kernel giving sharper spectral features than Jackson, at the cost of <code>m_order</code> extra parameters.

High-Level LDOS Dispatch (TBHamiltonian interface)

Signature	Description
<code>get_ldos(H, ω_phys; mode=:diag, psi0=nothing, kernel=:jackson, lambda=4.0, eta=0.0, m_order=4, maxdim=40, cutoff=1e-8, zl, wl) $\rightarrow$ MPS or MPO or Real</code>	Unified LDOS dispatch. Requires a prior <code>KPM.Tn</code> call to populate the cache. <code>mode=:diag</code> (default): returns diagonal MPS (requires MPO cache). <code>mode=:mpo</code> : returns full spectral MPO (requires MPO cache; legacy). <code>mode=:mps</code> : returns Real from moments (requires MPS cache + <code>psi0</code>).
<code>get_ldos_spectrum(H, ω_phys_vals; kernel=:jackson, lambda=4.0, maxdim=40, cutoff=1e-8) $\rightarrow$ Vector{Union{Nothing,MPS}}</code>	Site-resolved LDOS at all energies in a single pass over the cached MPO list. Real-space analogue of <code>get_bands</code> . Requires MPO cache.
<code>get_ldos_online(H, Ncheb, X, ω_phys_vals; kernel=:jackson, lambda=4.0, maxdim=100, cutoff=1e-8, verbose=false) $\rightarrow$ Vector{Float64}</code>	Online LDoS at 1-indexed site X for all energies, using only 3 MPS simultaneously. Never stores any Chebyshev cache. Works for standard and exciton Hamiltonians (exciton: evaluation state is $ X, X\rangle$).

```

get_ldos_spatial(H::TBHamiltonian,
Ncheb,  $\omega_{\text{phys\_vals}}$ ; num_x=H.N,
num_y=nothing, num_avg=1, mode=:mpo,
x_start=1, x_end=H.N,
x_groups=nothing, grid=false,
xwin=nothing, ywin=nothing,
box_half=0, reduce=:point,
sublattice=:auto, kernel=:jackson,
lambda=4.0, maxdim=100, cutoff=1e-8,
verbose=false, nambu_proj=false,
proj_nambu=nothing, spin_proj=false,
proj_s=nothing, layer_proj=false,
proj_layer=nothing,
sublat_proj=false, proj_sl=nothing) →
Matrix{Float64}

```

Spatially-resolved LDOS, real-space analogue of `get_bands`; returns an $N_\omega \times N_x$ matrix. Sample positions/blocks come from `spatial_sampling_plan` (built from `H.L`, `H.Lx`, and the kwargs below).

`reduce=:point` (default): read the LDOS at `num_x[×num_y]` sample cells (`box_half>0` averages a $(2 \text{ box_half} + 1)^2$ box); cheap but can alias features narrower than the grid.

`reduce=:block` (`mode=:mpo` only): partition into `num_x×num_y` power-of-two blocks and report the block integral via partial contraction — gap-free, catches thin in-gap/edge states. `grid=true` lays centers on a 2D `num_x×num_y` unit-cell grid; `xwin/ywin` give 0-indexed unit-cell window bounds for zooming into a patch.

`sublattice` controls the column layout when `H.sublattice_s` is set: `:auto` (default) resolves per-atom columns ($N_\omega \times (n_g n_{\text{sub}})$, atom order $[A_0, B_0, A_1, B_1, \dots]$) at full unit-cell resolution and averages sublattices into one column per cell ($N_\omega \times n_g$) for coarser grids or `box_half>0`; `:resolve/average` force one or the other, and `proj_sl=k` always resolves sublattice k alone. `mode=:mpo` (default): single Chebyshev pass, cost independent of `num_x/n_sub`. `mode=:mps`: independent MPS recursion per (position, sector). `nambu_proj/proj_nambu`, `spin_proj/proj_s`, `layer_proj/proj_layer`, `sublat_proj/proj_sl`: same auxiliary-DOF projection interface as `get_bands`; auto-enabled when the corresponding aux index is present.


```
get_dos_stochastic(H::TBHamiltonian,
Ncheb,  $\omega_{\text{phys\_vals}}$ ; N_sample=50,
N_bound=0, seed=42, normalize=false,
dos_weighting=:trace,
kernel=:jackson, lambda=4.0, eta=0.0,
m_order=4, maxdim=100, cutoff=1e-8,
verbose=false, nambu_proj=false,
proj_nambu=nothing, spin_proj=false,
proj_s=nothing, layer_proj=false,
proj_layer=nothing,
sublat_proj=false, proj_sl=nothing) →
Vector{Float64}
```

Stochastic full DoS via random trace estimation (MPS Chebyshev, 3 MPS per sample). `dos_weighting=:trace` (default) returns the trace DoS $\text{Tr}[\delta(\omega - H)]$, divided by the traced dimension if `normalize=true`; `dos_weighting=:sample` returns the raw sample-averaged signal (useful to inspect an exciton bound peak before sector-size weighting), and `normalize` is then ignored. Reconstruction uses `_dos_weight_matrix`: convolution kernels (`:jackson` default, `:lorentz` via `lambda`, `:fejer`, `:dirichlet`) or `kernel=:hodc` (`eta`, `m_order`; `eta=0` → $1/(N_{\text{cheb}}+1)$). Unlike `get_ldos_spatial`, auxiliary-DOF projections (`nambu_proj`, `layer_proj`, `sublat_proj`, ...) are *not* auto-enabled — the default samples the full D -dimensional Hilbert space; setting any `*_proj=true` samples position states with fixed aux sectors ($N_{\text{phys}} \times n_{\text{sectors}}$ effective states). For exciton Hamiltonians (`length(H.sites)==2H.L`) with no aux projections, `N_bound>0` dedicates samples to the bound sector $|x, x\rangle$, combined as $(D - N_{\text{phys}})\bar{A}_{\text{full}} + N_{\text{phys}}\bar{A}_{\text{bound}}$; `N_bound=0` (default) is uniform sampling over all D states.

Spatial Exciton LDOS

Signature	Description
<pre>get_exciton_ldos_spatial(H::TBHamiltonian, Ncheb, $\omega_{\text{phys_vals}}$; X_list=nothing, X_groups=nothing, x_groups=nothing, num_x=H.N, num_avg=1, x_start=1, x_end=H.N, kernel=:jackson, lambda=4.0, eta=0.0, m_order=4, maxdim=100, cutoff=1e-8, verbose=false, printinfo=false) → Matrix{Float64}</pre>	Spatially-resolved exciton LDOS. For each bound exciton position X (electron = hole = X, 1-indexed in $1:H.N$), runs an online MPS Chebyshev recursion from $X, X\rangle$ and accumulates all requested energies in one pass; no Chebyshev cache is stored on H. Rows are energies, columns are positions/groups. <code>X_list</code> selects positions directly; <code>X_groups</code> (alias <code>x_groups</code>) averages several bound-pair probes into one output column. If neither is given, <code>num_x</code> coarse groups are generated over <code>x_start:x_end</code> with <code>num_avg</code> sub-positions per group. <code>kernel=:hodc</code> uses the HODC reconstruction (<code>eta</code>, <code>m_order</code>); otherwise <code>:jackson/:lorentz/:fejer/:dirichlet</code>.

<code>get_exciton_ldos(H::TBHamiltonian, X::Int, omega_phys::Real; Ncheb=200, kernel=:jackson, lambda=4.0, eta=0.0, m_order=4, maxdim=40, cutoff=1e-8, verbose=false) → Real</code>	Single-position convenience wrappers around <code>get_exciton_ldos_spatial</code> with <code>X_list=[X]</code> : the scalar overload returns <code>ldos[1,1]</code> for one energy, the vector overload returns <code>vec(ldos[:,1])</code> for a list of energies.
<code>get_exciton_ldos(H::TBHamiltonian, X::Int, omega_phys_vals; ...) → Vector{Float64}</code>	

`_get_exciton_ldos_cached` is a legacy cache-backed scalar helper (Chebyshev moments via KPM.Tn); the public exciton LDOS API above routes through `get_exciton_ldos_spatial` instead, which performs online MPS recursion without storing a cache on `H`.

HODC Kernel

Signature	Description
<code>compute_hodc_params(m=6) → (z1, w1)</code>	Compute m quadrature nodes <code>z1</code> and weights <code>w1</code> for the HODC kernel. Call once; pass to KPM functions as <code>kernel=:hodc</code> , <code>z1=z1</code> , <code>w1=w1</code> .

Density Matrix Purification (Purification.tk.jl)

Iterative methods that drive eigenvalues of an approximate density matrix to exactly 0 or 1, refining the zero-temperature projector $\theta(\mu - H)$ without Gibbs ringing.

Initial Guess

Signature	Description
<code>purification.initial_guess(H_mpo, scale, sites; maxdim=40, cutoff=1e-8) → MPO</code>	Low-level form. Build $\rho_0 = \frac{1}{2}I - \frac{1}{2s}H$. Works only for center-symmetric spectra.
<code>purification.initial_guess(H::TBHamiltonian, $\epsilon_F=0.0$, maxdim=40, cutoff=1e-8) → MPO</code>	High-level form. Calls <code>_ensure_scale!</code> automatically. Uses formula accounting for nonzero center and Fermi-level shift ϵ_F : $\rho_0 = \left(\frac{1}{2} + \frac{\epsilon_F + \text{center}}{2\text{scale}}\right)I - \frac{H}{2\text{scale}}.$

McWeeny Purification

Signature	Description
<code>mcweeny_purify(ρ_0; maxiters=30, maxdim=40, cutoff=1e-8, tol=1e-5, verbose=false) → MPO</code>	Iterate $\rho_{n+1} = 3\rho_n^2 - 2\rho_n^3$ until $\ \rho^2 - \rho\ /\ \rho\ < \text{tol}$. Quadratic convergence; 2 MPO products per step.
<code>mcweeny_purify(H::TBHamiltonian; $\epsilon_F=0.0$, maxiters=30, maxdim=40, cutoff=1e-8, tol=1e-5, verbose=false) → MPO</code>	High-level form. Builds initial guess from <code>H</code> (optionally shifted by ϵ_F), runs McWeeny, caches result in <code>H.density_cache</code> .

SP2 Purification

Signature	Description
<code>sp2_purify(ρ_0, N_{el}; maxiters=40, maxdim=40, cutoff=1e-8, tol=1e-5, verbose=false) → MPO</code>	Iterate: if $\text{Tr}(\rho^2) \geq N_{\text{el}}$: $\rho_{n+1} = \rho^2$; else $\rho_{n+1} = 2\rho - \rho^2$. Drives $\text{Tr}(\rho) \rightarrow N_{\text{el}}$; 1 MPO product per step.
<code>sp2_purify(H::TBHamiltonian; $N_{\text{el}}=H.N \div 2$, maxiters=40, maxdim=40, cutoff=1e-8, tol=1e-5, verbose=false) → MPO</code>	High-level form. N_{el} defaults to half-filling. Caches result in <code>H._density_cache</code> .

Unified Density-Matrix Wrapper

Signature	Description
<code>get_density(H::TBHamiltonian; method=:mcweeny, $\epsilon_F=0.0$, Ncheb=150, kernel=:jackson, lambda=4.0, maxdim=40, cutoff=1e-8, $N_{\text{el}}=H.N \div 2$, maxiters=30, tol=1e-5, verbose=false) → MPO</code>	Compute and cache the zero-temperature density matrix $P = \theta(\epsilon_F - H)$. Returns <code>H._density_cache</code> immediately if already populated (set it to <code>nothing</code> to force recomputation). <code>method</code> : <code>:mcweeny</code> (McWeeny purification, default), <code>:sp2</code> (SP2 purification, requires N_{el}), or <code>:kpm</code> (KPM Chebyshev expansion of the Fermi step function, reusing/building <code>H._tn_cache</code> at order <code>Ncheb</code> ; only <code>:jackson</code> kernel is meaningful for the step function).

Sign of an MPO

Signature	Description
<code>sign_mpo(A::MPO, sites; scale=1.0, maxdim=500, cutoff=1e-8, maxiters=30, tol=1e-5, verbose=false) → MPO</code>	Compute $\text{sign}(A) = \theta(A) - \theta(-A)$ for a Hermitian MPO <code>A</code> via two McWeeny purifications of $\rho_{0\pm} = (I \pm A/\text{scale})/2$. <code>scale</code> must bound the spectral radius of <code>A</code> (default 1.0, correct for operators with spectrum in $(-1, +1)$ such as <code>get_valley_operator</code>). Result has eigenvalues in $\{-1, +1\}$.

LDoS and DoS via Density-Matrix Derivative

Signature	Description
-----------	-------------

<code>get_ldos_drho(H::TBHamiltonian, ω; mode=:mpo, dmu=0.05, maxdim=40, cutoff=1e-8, maxiters=30, tol=1e-5, verbose=false) → MPO or MPS</code>	Local density of states $A(\omega) \approx [\rho(\omega + \delta\mu) - \rho(\omega - \delta\mu)]/(2\delta\mu)$ via finite-difference McWeeny density matrices, $\rho(\mu) = \theta(\mu - H)$. <code>mode=:mpo</code> (default) returns the full MPO $(LDoS(r, \omega) = \langle r A(\omega) r \rangle)$; <code>mode=:mps</code> returns only the diagonal as an MPS (cheaper). A vector overload <code>get_ldos_drho(H, ωs::AbstractVector; ...)</code> applies this elementwise.
<code>get_dos_drho(H::TBHamiltonian, ω; dmu=0.05, maxdim=40, cutoff=1e-8, maxiters=30, tol=1e-5, verbose=false) → Float64</code>	Total density of states $DOS(\omega) \approx$ $(\text{Tr}[\rho(\omega + \delta\mu)] - \text{Tr}[\rho(\omega - \delta\mu)])/(2\delta\mu)$. Cheapest variant: only scalar traces are computed. A vector overload <code>get_dos_drho(H, ωs::AbstractVector; ...) → Vector{Float64}</code> applies this elementwise.

Quantum Fourier Transform and Band Structure (QFT_tk.jl)

QFT Conjugation

Signature	Description
<code>conjugate_by_qft(W; tol=1e-9, maxdim=100) → MPO</code>	Returns $UWU^\dagger$ where U is the normalised QFT MPO (LSB at site 1). Maps any real-space MPO to momentum space.
<code>conjugate_by_qft(H::TBHamiltonian, W::MPO; tol=1e-9, maxdim=100) → MPO</code>	TBHamiltonian-aware overload. Applies $UWU^\dagger$ where U acts only on the position (Qubit) sites of H , with identity at every auxiliary site (layer, spin, sublattice, Nambu). Use when W lives on the full H .sites space, e.g. after <code>replace_sites</code> .
<code>conjugate_by_qft_exciton(H::TBHamiltonian, W; tol=1e-9, maxdim=100) → MPO</code>	Fermion, particle (electron-hole) analogue for the interleaved 2L-site exciton encoding (<code>exciton_hamiltonian</code>). Applies $UWU^\dagger$ with $U = U_e \otimes U_h$ — two independent QFTs, one on the electron register (odd sites) and one on the hole register (even sites), $R = H.L$ qubits each. Sites are taken from <code>_pos_sites(H)</code> . Rotates an exciton spectral MPO T_n into (k_e, k_h) momentum space for band structure; pair with an <i>MPS-mode</i> KPM, since the MPO-mode Chebyshev list explodes in bond dimension for excitons.

Exciton Momentum-Space Spectra (MPS-KPM)

Momentum-resolved exciton spectra use the MPS mode end to end: conjugate the exciton Hamiltonian with `conjugate_by_qft_exciton`, build a fixed-total-momentum probe state, and run an online MPS Chebyshev recursion against the QFT'd MPO. The MPO-mode `get_bands` pipeline must *not* be used here — the exciton Chebyshev-MPO bond dimension explodes. All

momenta are 1-indexed ($1:2^L$), LSB-first, with electron bits on odd sites and hole bits on even sites.

Momentum-basis probe states. Compact finite-state MPS on the interleaved $2L$ -qubit chain, each built in $O(L)$ via a carry bond that enforces $k + h \equiv Q \pmod{N}$, never by explicit summation over the $N = 2^L$ momenta.

Signature	Description
<code>mpsexcitonQ(Q, sites) → MPS</code>	Coherent fixed-total-momentum pair state $ Q\rangle = N^{-1/2} \sum_k k, -k + Q\rangle$. The reference probe for exciton <i>bands</i> .
<code>mpsexcitonQTrace(Q, sites; rng) → MPS</code>	Random-phase (± 1) trace probe $ r_Q\rangle = N^{-1/2} \sum_k \eta_k k, Q-k\rangle$ with $E[\eta_k \eta_l] = \delta_{kl}$, so averaging probes estimates the fixed- Q continuum trace. The stochastic <i>continuum</i> probe.
<code>mpsexcitonKQ(k, Q, sites) → MPS</code>	Single product basis vector $ k, Q-k\rangle$ in the fixed- Q sector; the deterministic continuum probe sampled by <code>get_exciton_continuum</code> .

Spectral drivers. MPS-KPM analogues of `get_bands`: one online Chebyshev recursion per momentum (or averaged group), accumulating all energies in a single pass. `H_QFT` supplies the conjugated MPO; the real-space `H::TBHamiltonian` supplies metadata (`sites`, `N`, `L`, `scale`, `center`). Rows are energies, columns are total momenta.

Signature	Description
<code>get_exciton_bands(H_QFT, H, Ncheb, ω_phys_vals; Q_list, Q_groups, ...) → Matrix{Float64}</code>	Coherent exciton bands $A(Q, \omega)$ from the <code>mpsexcitonQ</code> pair state. <code>Q_list</code> selects momenta; <code>Q_groups</code> averages several probes into one column; otherwise a window (<code>q_start/q_end/num_q/num_avg</code>) is sampled. The <code>K_*/k_*</code> kwargs are backward-compatible aliases.
<code>get_exciton_continuum(H_QFT, H, Ncheb, ω_phys_vals; Q_list, N_sample, k_list, ...) → Matrix{Float64}</code>	Incoherent continuum $A_{\text{cont}}(Q, \omega) = N^{-1} \sum_k \langle k, Q-k \delta(\omega - H) k, Q-k \rangle$ via <code>mpsexcitonQTrace</code> random-phase traces (or a deterministic <code>k_list</code>). <code>N_sample</code> sets the number of trace probes per Q . <code>normalize=true</code> (default) returns the per-relative-momentum average; <code>false</code> returns the total fixed- Q trace ($\times H.N$).

Online Band Structure: `get_bands`

`get_bands` runs an online Chebyshev KPM recurrence over the full MPO space without ever materialising the full T_n list. It replaces the legacy inner-product approach and requires no prior `KPM.Tn` call.

Signature	Description
-----------	-------------

<code>get_bands(H_mpo, scale, center, sites, Ncheb, D, ω_vals; ...) $\rightarrow$ Matrix{Float64}</code>	Low-level form. ω _vals must be in rescaled units $\in (-1, 1)$. $D=1$ or 2 sets the spatial dimension. Returns $N_\omega \times N_k$ matrix.
<code>get_bands(H::TBHamiltonian, Ncheb, D, ω_phys_vals; ...) $\rightarrow$ Matrix{Float64} or NamedTuple</code>	High-level form. Physical energies rescaled via <code>H.scale/H.center</code> . All auxiliary indices auto-detected from struct fields; no manual index passing. Returns <code>NamedTuple(Ak, ticks, labels)</code> when <code>kpath</code> kwarg is used, otherwise <code>Matrix{Float64}</code> .
<code>get_bands(H::TBHamiltonian, Ncheb, ω_phys_vals; ...) $\rightarrow$ Matrix{Float64} or NamedTuple</code>	Convenience form; infers D from <code>length(H.geometry(1))</code> . Errors if <code>H.geometry</code> is <code>nothing</code> .

Five-step projection pipeline (all steps optional and composable):

#	Step	Kwargs	Notes
0	Nambu (BdG)	<code>nambu_proj</code> , <code>proj_nambu</code>	Sectors 1=particle, 2=hole.
1	Spin	<code>spin_proj</code> , <code>proj_s</code> , <code>spin_s_aux</code>	Sectors 1= $\uparrow$, 2= $\downarrow$.
1c	Layer	<code>layer_proj</code> , <code>proj_layer</code>	For bilayer/multilayer models.
1b	Sublattice (aux)	<code>sublat_proj</code> , <code>proj_sl</code> , <code>sublat_s</code>	For kagomé, Lieb, honeycomb, dice (<code>H.sublattice_s</code> set).
2	Sublattice (legacy)	<code>sublattice</code> , <code>proj_sl</code>	Mask sandwich for preset models without <code>sublattice_s</code> .

`proj_*=nothing` sums over all sectors; `proj_*=k` selects sector k .

k-point sampling kwargs: `xmin`, `xmax`, `num_x` (grid in k_x); `ymin`, `ymax`, `num_y` (2D); `num_avg` (points averaged per center); `k.groups_override` (explicit path from `kpath.2d`).

High-symmetry k-path shortcut:

```
res = get_bands(H, Ncheb, 2, omega;
               kpath=[:G, :M, :Kp, :G], kpath_lattice=:honeycomb, num_x=30)
# res.Ak -- (N_omega x N_k) matrix
# heatmap(1:size(res.Ak,2), omega, res.Ak; xticks=(res.ticks, res.labels))
```

High-Symmetry k-Path Utilities

Signature	Description
<code>hsk_honeycomb(Lx, Ly) $\rightarrow$ NamedTuple</code>	High-symmetry k-points for the honeycomb sublattice Hamiltonian in quantic integer units. Fields: $G=(0,0)$, $M=(N_x/2, N_y/4)$, K , Kp (Dirac points, rounded to nearest integer).
<code>hsk_square(Lx, Ly) $\rightarrow$ NamedTuple</code>	High-symmetry points for square lattice: G , $X=(N_x/2, 0)$, $M=(N_x/2, N_y/2)$.
<code>hsk_triangular(Lx, Ly) $\rightarrow$ NamedTuple</code>	Approximate high-symmetry points for triangular lattice: G , M , K .

<code>kpath_2d(hs_points, Lx; npts_per_segment=20) → (k_groups, tick_positions)</code>	Interpolate between <code>(kx_idx, ky_idx)</code> tuples; returns <code>k_groups</code> for <code>k_groups_override</code> and <code>tick_positions</code> for plot axes.
<code>kpath_setup(lattice, Lx, Ly, path_syms; npts_per_segment=20) → (k_groups, ticks, labels)</code>	Build k-path from symbol list (e.g. <code>[:G, :M, :Kp, :G]</code>) for given lattice (<code>:honeycomb</code> , <code>:square</code> , <code>:triangular</code>). Labels use Unicode: $G \rightarrow \Gamma$, $Kp \rightarrow K'$. Called internally by <code>get_bands</code> <code>kpath</code> shortcut.

Auxiliary Index Projection Helpers

Signature	Description
<code>project_aux(W, aux_s, sigma; side=:pre) → MPO</code>	Remove one auxiliary site from L -site MPO W by projecting onto state σ . Returns $(L - 1)$ -site position-only MPO. <code>side=:pre</code> (prepended) or <code>:post</code> (postpended).
<code>project_spin(W, spin_s, sigma) → MPO</code>	Convenience alias for <code>project_aux(W, spin_s, sigma; side=:pre)</code> .
<code>aux_site(H::TBHamiltonian, which) → (Index, Symbol)</code>	Return the auxiliary Index and its side (<code>:pre</code> / <code>:post</code>) for <code>which</code> $\in$ <code>{:spin, :nambu, :layer, :sublattice}</code> . Auto-called by the <code>TBHamiltonian</code> overload of <code>get_bands</code> .

Note: `get_spect_k` and `get_spect_k_doubled` (inner-product approach) have been superseded by the diagonal-extraction approach in `get_bands` and are no longer part of the public API.

Topological Invariants (Topology_tk.jl)

Real-space topological invariants (Chern number and winding number) for general lattice geometries. All functions accept user-supplied coordinate functions `xfunc(i, L_chain)` and `yfunc(i, L_chain)` where i is a 0-indexed site number and $L_chain=2^l$ is the relevant lattice length.

Key concept — quenching: Periodic boundary conditions would make $\hat{x}$ discontinuous. With `quenched=true` the position operators are replaced by $\sin(x/\Lambda)$ and $\cos(x/\Lambda)$, which are bounded and smooth. The identity $x \approx \Lambda \sin(x/\Lambda)$ for small arguments means the quenched Chern marker recovers physical units via a factor Λ^2 . The winding-number density recovers via Λ .

Projector Retrieval

Signature	Description
-----------	-------------

`_get_projector(H; method=:KPM,
fermi=0.0, Nchebychev=300, maxdim=40,
cutoff=1e-8, Nel=nothing) → MPO`

Compute or retrieve the zero-temperature projector $P = \theta(\mu - H)$. `method=:KPM`: uses cached `H._tn_cache` or runs KPM; `fermi` in physical units. `method=:mcweeny`: uses `H._density_cache` or calls `mcweeny_purify`. `method=:sp2`: uses `H._density_cache` or calls `sp2_purify` with target `Nel`.

Quenched Position Operators

These build diagonal MPOs for the trigonometric quenching functions. The argument to `get_diagonal_mpo` is 1-indexed; these builders convert internally with `xfunc(i-1, L_chain)`.

Signature	Description
<code>get_sinx_op(L, sites, L_chain, Λ, xfunc) → MPO</code>	Diagonal MPO for $\sin(x_i/\Lambda)$.
<code>get_cosx_op(L, sites, L_chain, Λ, xfunc) → MPO</code>	Diagonal MPO for $\cos(x_i/\Lambda)$.
<code>get_siny_op(L, sites, L_chain, Λ, yfunc) → MPO</code>	Diagonal MPO for $\sin(y_i/\Lambda)$.
<code>get_cosy_op(L, sites, L_chain, Λ, yfunc) → MPO</code>	Diagonal MPO for $\cos(y_i/\Lambda)$.

1D Winding Number

Signature	Description
<code>get_W(H::TBHamiltonian, xfunc=nothing; method=:KPM, fermi=0.0, Nchebychev=300, maxdim=15, cutoff=1e-8, Nel=nothing, quenched=true, l=nothing, $\Lambda=10$) → Function</code>	Compute the winding-number density operator. <code>H</code> must have a 2-component sublattice index (<code>H.sublattice_s</code> , <code>dim 2</code>); $\sigma_z = \text{diag}(+1, -1)$ on that index is built automatically. <code>xfunc(i, L_chain)</code> (0-indexed) defaults to <code>nothing</code> , auto-derived from <code>H.geometry_uc</code> (preferred) or <code>H.geometry</code> . Pre-computes one or two MPOs (W_1, W_2 for <code>quenched=true</code>). Returns a closure <code>calculate_winding(uc::Int) -> ComplexF64</code> where <code>uc</code> is a 1-indexed unit-cell number ($1 \dots 2^L$); the value sums the winding marker over both sublattice atoms (A and B) in that cell.

Quenched case (`quenched=true`): Uses the 2-term decomposition $\langle \alpha | \hat{W} | \alpha \rangle = \Lambda [\cos(x_\alpha/\Lambda) \langle \alpha | W_1 | \alpha \rangle - \sin(x_\alpha/\Lambda) \langle \alpha | W_2 | \alpha \rangle]$ where $W_1 = \sigma_z(P \sin \hat{X} Q + Q \sin \hat{X} P)$, $W_2 = \sigma_z(P \cos \hat{X} Q + Q \cos \hat{X} P)$, and α ranges over the two sublattice sites (A, B) of unit cell `uc`.

Flat case (`quenched=false`): builds $W = \sigma_z(P \hat{x} Q + Q \hat{x} P)$ once; the closure evaluates $\sum_{\alpha \in \{A, B\}} \langle \alpha | W | \alpha \rangle$ for unit cell `uc`.

2D Real-Space Chern Marker

Signature	Description
<code>get_C_op_MPO_from_P(P, L, sites, xfunc, yfunc; l=nothing, Λ=10, maxdim=500, cutoff=1e-8, quenched=true, sequential=false, pk_mpo=nothing) → Function</code>	Build real-space Chern marker from pre-computed projector P . sequential=true avoids forming the 4 α -independent C_1 – C_4 MPOs, instead applying operators directly to each basis MPS (fewer MPO-MPO products, more inner products per site). pk_mpo optionally sandwiches a projector (e.g. a valley projector from <code>get_valley_C</code>) into the marker: $\langle \alpha P_K C P_K \alpha \rangle$. Returns closure <code>calculate_chern_number(uc::Int) → ComplexF64</code> , uc a 1-indexed unit-cell number ($1 \dots 2^L$), summed over sublattice atoms.
<code>get_C(H::TBHamiltonian, xfunc=nothing, yfunc=nothing; method=:KPM, fermi=0.0, l=nothing, Λ=10, Lambda=nothing, Nchebychev=300, maxdim=500, cutoff=1e-8, Nel=nothing, quenched=true, sequential=false) → Function</code>	High-level wrapper: calls <code>_get_projector</code> then <code>get_C_op_MPO_from_P</code> . xfunc/yfunc default to <code>nothing</code> and are auto-derived from <code>H.geometry_uc</code> (sublattice models) or <code>H.geometry</code> . Lambda is an ASCII alias for Λ . Returns same <code>calculate_chern_number</code> closure.

Quenched case (`quenched=true`): uses the 4-term decomposition

$$\begin{aligned} \langle \alpha | \hat{C} | \alpha \rangle = 2\pi i \Lambda^2 & \left[\cos \frac{x_\alpha}{\Lambda} \cos \frac{y_\alpha}{\Lambda} \langle \alpha | C_1 | \alpha \rangle + \sin \frac{x_\alpha}{\Lambda} \sin \frac{y_\alpha}{\Lambda} \langle \alpha | C_2 | \alpha \rangle \right. \\ & \left. - \cos \frac{x_\alpha}{\Lambda} \sin \frac{y_\alpha}{\Lambda} \langle \alpha | C_3 | \alpha \rangle - \sin \frac{x_\alpha}{\Lambda} \cos \frac{y_\alpha}{\Lambda} \langle \alpha | C_4 | \alpha \rangle \right] \end{aligned}$$

where C_1 – C_4 are pre-computed α -independent MPOs. Only scalar trig evaluations are needed per site.

Flat case (`quenched=false`): $\hat{C} = 2\pi i (Q\hat{x}P\hat{y}Q - P\hat{x}Q\hat{y}P)$.

Valley Operator and Valley Chern Number (Honeycomb)

Signature	Description
<code>get_valley_operator(H::TBHamiltonian; maxdim=500, cutoff=1e-8) → MPO</code>	Build the valley operator V for a 2D honeycomb Hamiltonian (<code>H.Lx</code> set, 2-component <code>H.sublattice_s</code>). V is the Haldane next-nearest-neighbour Hamiltonian ($\varphi = \pi/2$, NN term zeroed) multiplied by a sublattice sign η_i (+1 on A, −1 on B), with global prefactor $-i/(3\sqrt{3})$.
<code>get_valley_projectors(V_mpo::MPO, sites; maxdim=500, cutoff=1e-8) → (PK, PK_prime)</code>	Return the two valley projectors $P_K = (I + V)/2$ and $P_{K'} = (I - V)/2$ from valley operator <code>V_mpo</code> .

<pre>get_valley_C(H::TBHamiltonian, xfunc=nothing, yfunc=nothing; valley=:K, use.sign=true, method=:mcweeny, fermi=0.0, l=nothing, Λ=10, Nchebychev=300, maxdim=500, cutoff=1e-8, Nel=nothing, quenched=true, sequential=false) → Function</pre>	Valley-resolved Chern marker. Builds $V = \text{get_valley_operator}(H)$; if <code>use.sign=true</code> (default) sharpens it to $S = \text{sign}(V) \in \{-1, +1\}$ via <code>sign_mpo</code>. Forms $P_K = P(I \pm S)/2$ (<code>sign +</code> for <code>valley=:K</code>, <code>-</code> for <code>:K_prime</code>), symmetrizes it, and passes it as <code>pk_mpo</code> to <code>get_C_op_MPO_from_P</code>, which evaluates $\langle \alpha P_K C P_K \alpha \rangle$. Returns closure <code>calculate_valley_chern(uc::Int) -> ComplexF64</code>.
---	---

Thouless Pump

Computes the Thouless pump Chern number via the M1Q marker: $C = \text{M1Q}(T) - \text{M1Q}(0)$, where $\text{M1Q}(r, t) = \langle r | P(t) U^\dagger(t) \hat{x} U(t) P(t) | r \rangle$ and $U(t)$ is the adiabatic evolution operator propagated by $h(t) = [\dot{P}, P]$ via a second-order Taylor step.

Signature	Description
<pre>get_pump_xop(L, sites, xfunc; quenched=false, Λ=-1.0) → MPO</pre>	Build the position operator MPO for Thouless pump. <code>quenched=false</code>: diagonal entries are <code>xfunc(i, N)</code> directly. <code>quenched=true</code>: entries are $\Lambda \sin(x/\Lambda)$ to smooth the PBC discontinuity (Λ defaults to $N = 2^L$).
<pre>thouless_pump(P_array, dt, x_op, sites; r_center, maxdim=100, cutoff=1e-8, verbose=false, return_trajectory=false) → Float64 or (Float64, Vector{Float64})</pre>	Compute C from a pre-built array of projectors <code>P_array</code> (length <code>Nt</code>). <code>r_center</code>: 0-indexed bulk site at which M1Q is evaluated. With <code>return_trajectory=true</code> returns $(C, \text{M1Q_traj})$ where <code>M1Q_traj</code> has length <code>Nt+1</code> (index $1 = t = 0$).
<pre>get_thouless_pump(H_of_t, Nt, T, xfunc; P_method=:mcweeny, fermi=0.0, Nchebychev=200, maxdim=100, cutoff=1e-8, quenched=false, Λ=-1.0, Nel=nothing, r_center=nothing, verbose=false) → Float64</pre>	High-level wrapper. Calls <code>H_of_t(t_k)</code> to obtain the Hamiltonian at each of the <code>Nt</code> time steps, builds $P(t_k)$ via <code>P_method</code> (<code>:mcweeny</code>, <code>:sp2</code>, or <code>:KPM</code>), then calls <code>thouless_pump</code>. <code>r_center</code> defaults to $2^L/2$.

Typical usage:

```
# Factory: same site indices must be reused for all t
H0 = get_Hamiltonian("chain_1d", 1.0; L=8)
ref_sites = H0.sites
H_of_t(t) = get_Hamiltonian("ssh", (t=1.0, d=0.5*cos(2*pi*t/T));
                           L=8, ref_sites=ref_sites)

xfunc(i, N) = Float64(div(i, 2)) # x-coord for SSH (two sites per unit cell)
C = get_thouless_pump(H_of_t, 40, T, xfunc;
                     P_method=:mcweeny, verbose=true)
println("Thouless pump invariant: ", round(C; digits=3))
```

DMRG (DMRG_tk.jl)

Signature	Description
<code>dmrg_gs(H, sites; nsweeps=10, maxdim=[10,20,50,100,200], cutoff=1e-8, noise=[...], linkdim_init=10) → (E, ψ)</code>	Ground state and energy of H via DMRG.
<code>build_K(H_mpo, sites, ω, η; maxdim_K=200, cutoff_K=1e-8) → MPO</code>	Resolvent-squared MPO $K(\omega, \eta) = (H - \omega I)^2 + \eta^2 I$. Ground-state energy $E_K \geq \eta^2$; $E_K \rightarrow \eta^2$ at spectral peaks.
<code>dmrg_spectral(H_mpo, sites, ω, η; ψ_0=nothing, ...) → (E_K, ψ)</code>	DMRG ground state of $K(\omega, \eta)$. Output $ \psi(\omega)\rangle$ concentrates on the eigenstate of H nearest ω .
<code>local_weight(ψ, i, L, sites) → Float64</code>	$ \langle i \psi\rangle ^2$: LDoS proxy at site i (0-indexed, big-endian). Use with <code>dmrg_spectral</code> output.

Time Evolution (Timeev_tk.jl)

Propagator MPO and Single-Step TDVP

Signature	Description
<code>tdvp_evolve(H, ψ, dt; maxdim=200, cutoff=1e-10, normalize=true, nsite=2) → MPS</code>	Single TDVP step; thin wrapper around <code>ITensorMPS.tdvp</code> .
<code>build_tdvp_propagator_mpo(H, dt, L, sites; maxdim=50, cross_tol=1e-8, ...) → MPO</code>	Compress $U(\Delta t) = e^{-iH\Delta t}$ into an MPO via QTCI on matrix elements $\langle i U j\rangle$ evaluated by TDVP. Diagonal pivots seeded by default. <code>TBHamiltonian</code> overload available.
<code>apply_mpo_to_mps(U_mpo, ψ; cutoff=1e-12, maxdim=500, normalize=true) → MPS</code>	Apply propagator MPO to a single MPS state; optionally normalize.

Trajectory Evolution: MPS States

Signature	Description
<code>evolve_with_propagator(U_mpo, ψ_0, nsteps; normalize_each_step=true, cutoff=1e-8, maxdim=10000) → Vector{MPS}</code>	Apply the same propagator MPO <code>nsteps</code> times; returns length-(<code>nsteps</code> +1) trajectory including the initial state.
<code>evolve_with_tdvp(H, ψ_0, nsteps, dt; normalize_each_step=true, maxdim=200, cutoff=1e-10, ...) → Vector{MPS}</code>	Multi-step TDVP trajectory for time-independent Hamiltonian. Returns full trajectory [<code>psi(0)</code> , <code>psi(dt)</code> , ..., <code>psi(nsteps*dt)</code>]. <code>TBHamiltonian</code> overload: uses $-iH.mpo$.

```
evolve_with_tdvp_timedep(Hoft,  $\psi_0$ ,
nsteps, dt; normalize_each_step=true,
maxdim=200, cutoff=1e-10,
reverse_step=false, krylovdim=20,
tol=1e-10) → Vector{MPS}
```

TDVP trajectory for a time-dependent $H(t)$. Uses midpoint rule: H is frozen at $H(t + \Delta t/2)$ over each interval. `Hoft` is a callable `t → MPO` returning $H(t)$ itself (the function applies $-iH$ internally); no `TBHamiltonian` overload is provided for this function.

State Analysis Utilities

Signature	Description
<code>compute_basis_overlaps(states, L, sites) → NamedTuple</code>	Overlap of each MPS in <code>states</code> with all 2^L computational basis states. Returns (<code>overlaps</code> , <code>abs_overlaps</code> , <code>probabilities</code> , <code>norms</code>).
<code>basis_amplitude(ψ, n, L, sites) → ComplexF64</code>	Single-basis-state amplitude $\langle n \psi\rangle$ (0-indexed <code>n</code>).
<code>phase_aligned_distance(ψ_a, ψ_b) → Float64</code>	$\sqrt{2 - 2 \langle \hat{\psi}_a \hat{\psi}_b\rangle }$: distance minimised over global phase; 0 if states are equal up to phase and normalisation.
<code>check_tdvp_vs_U_mpo(H, U_mpo, dt, L, sites; ...) → (Float64, Float64)</code>	Validate <code>U_mpo</code> against direct TDVP on computational basis states. Returns (<code>max_overlap_error</code> , <code>max_distance</code>). <code>TBHamiltonian</code> overload available.

Density Matrix Dynamics (open / driven systems)

The density matrix ρ is represented as an MPO. The von Neumann equation $d\rho/dt = -i[H(t), \rho]$ is integrated with a classical RK4 scheme. Each $H(t)$ evaluation produces one MPO-MPO product, so bond dimension grows; use `maxdim` and `cutoff` to control cost.

Signature	Description
<code>rk4_step_dm_timedep(Hoft, ρ, t, dt; maxdim=200, cutoff=1e-10, truncate_intermediates=true) → MPO</code>	Single RK4 step for $d\rho/dt = -i[H(t), \rho]$. Evaluates H at the four RK4 sample times.
<code>evolve_rk4_dm_timedep(Hoft, ρ_0, nsteps, dt; maxdim=200, cutoff=1e-10, verbose=false) → Vector{MPO}</code>	Full RK4 trajectory; returns length-(<code>nsteps</code> +1) list including ρ_0 .
<code>dm_expect(O, ρ) → Float64</code>	Expectation value $\text{Tr}(O\rho)$ for Hermitian <code>O</code> . Uses <code>inner(O, rho)</code> .
<code>purity(ρ) → Float64</code>	$\text{Tr}(\rho^2) = \text{inner}(\rho, \rho)$. 1 for pure states, < 1 for mixed.
<code>purity_trajectory(states) → Vector{Float64}</code>	Apply purity to each MPO in <code>states</code> .

Observables Along Trajectories

Signature	Description
-----------	-------------

<code>observables_trajectory(ops, states::Vector{MPO}) → Dict</code>	Evaluate a named collection of MPO observables on a density-matrix trajectory. <code>ops</code> is a <code>DictString,MPO</code> ; returns <code>DictString,VectorFloat64</code> .
<code>timedep_observable_trajectory(OfT, states::Vector{MPO}, dt) → Vector{Float64}</code>	Evaluate a time-dependent observable <code>OfT(t) → MPO</code> at times $t_k = k \cdot \Delta t$ on the density-matrix trajectory.
<code>bond_current_x(ρ, j, tx, L, sites) → ComplexF64</code>	Bond current $J_j = 2 \text{Im}[t_x \rho_{j,j+1}]$ on the bond $(j, j+1)$ (0-indexed j). Uses off-diagonal matrix elements of ρ via <code>matrix_checker</code> .
<code>bond_current_x_trajectory(states, j, tx, L, sites; dt=1.0) → Vector{ComplexF64}</code>	Evaluate <code>bond_current_x</code> at every step of a trajectory.
<code>central_x_bond(L; Nx=nothing) → Int</code>	Return the 0-indexed site index of the central x -bond. For 2D ($L_x = L_y = L/2$) provides the horizontal bond nearest the geometric center; uses L_x rows if <code>Nx</code> is given.

Diagnostic Tools

Signature	Description
<code>compare_propagator_and_tdvp_heatmaps(U_mpo, H, ψ_0, L, sites, nsteps; ...) → NamedTuple</code>	Run both propagator-MPO and TDVP trajectories from the same initial state and return basis-state probability heatmaps for side-by-side comparison. <code>TBHamiltonian</code> overload available.

Two-Particle Systems (twoparticle_tk.jl)

Constructs the exciton (electron-hole) Hamiltonian on an interleaved $2L$ -site space.

Site encoding: L position qubits per sector, interleaved as $[e_1, h_1, e_2, h_2, \dots]$: odd sites $(1, 3, \dots)$ = electron, even sites $(2, 4, \dots)$ = hole. The Hamiltonian is $H_{\text{exc}} = H_c \otimes I_h - I_e \otimes H_v + U$ where U is a contact interaction diagonal in the electron-hole coincidence basis.

`TBHamiltonian.L` = position qubits per sector; `TBHamiltonian.sites` holds all $2L$ interleaved sites; `TBHamiltonian.N` = 2^L (single-sector physical sites).

High-Level Constructors

Signature	Description
<code>exciton_hamiltonian(geometry, params, Ufunc; L, on_site=nothing, scale=nothing, tol_quantics=1e-8, maxbonddim_quantics=100, tol=1e-8, cutoff=1e-8, maxdim=200, kwargs...) → TBHamiltonian</code>	Build an exciton Hamiltonian where both electron and hole sectors share the same <code>geometry/params</code> . Extra <code>kwargs</code> are forwarded to <code>get_Hamiltonian</code> .

<code>exciton_hamiltonian(H_c, H_v, Ufunc; on_site=nothing, scale=nothing, tol_quantics=1e-8, maxbondddim_quantics=100, tol=1e-8, cutoff=1e-8, maxdim=200) → TBHamiltonian</code>	Two-sector form for distinct electron/hole bands. <code>H_c</code> and <code>H_v</code> must have the same <code>L</code> and distinct site indices.
---	--

Keyword arguments (both forms):

- `Ufunc(x)` — contact interaction strength at site $x \in \{1, \dots, 2^L\}$ (1-indexed).
- `on_site` — conduction-band-edge modulation $V(x)$; added as $+V$ on electron, $-V$ on hole (type-I confinement: both carriers confined). Compressed via QTCL.
- `scale` — exciton half-bandwidth; 0.0 triggers lazy DMRG estimation on first KPM call.

Low-Level MPO Builders

Signature	Description
<code>Exciton_Hamiltonian(H_c, H_v, Ufunc; on_site, tol_quantics, maxbondddim_quantics, tol, cutoff, maxdim) → MPO</code>	Low-level builder returning a raw $2L$ -site MPO. Interleaves electron/hole kinetic terms and adds contact interaction.
<code>hopping_mpo_exciton(H_c, H_v) → MPO</code>	Embed single-particle sectors into the $2L$ -site space: $H_c \otimes I_h - I_e \otimes H_v$ without interaction.
<code>build_interaction_op_exciton(L, sites, Ufunc) → MPO</code>	Build the contact interaction diagonal MPO. <code>Ufunc(x)</code> on the diagonal where electron and hole coincide ($x_e = x_h = x$); off-diagonal elements are zero.

Typical usage:

```
# 1D exciton -- same geometry, contact interaction
H_exc = exciton_hamiltonian("chain_1d", 1.0, x -> -0.5;
                           L=8, on_site = x -> -0.1*exp(-((x-64)^2)/200))

# 2D exciton -- separate electron/hole bands
H_c = get_Hamiltonian("square_2d", 1.0; L=8, Lx=4, Ly=4)
H_v = get_Hamiltonian("square_2d", 0.5; L=8, Lx=4, Ly=4)
H_exc = exciton_hamiltonian(H_c, H_v, x -> -0.2)

# Use with standard TensorBinding tools
Tn, scale, center = KPM_Tn(H_exc, 200)
dos = real(tr(get_ldos_w_from_Tn(Tn, 200, 0.0; maxdim=200)))
```

Random Phase Approximation (RPA_tk.jl)

Computes the RPA susceptibility χ^{RPA} via the Dyson equation $(1 - \Pi_0 V)\chi = \Pi_0$, where Π_0 is the non-interacting (Lindhard) polarization bubble. The high-level `TBHamiltonian` API (`get_bubble_mpo`, `get_rpa_susceptibility`, and the magnon variants) is the recommended

entry point; the legacy `get_Tnlists / build_bubble_mpo` pipeline remains available as a lower-level building block.

High-Level TBHamiltonian API

Signature	Description
<code>get_bubble_mpo(H1::TBHamiltonian, H2::TBHamiltonian, ω; $\epsilon_F=0.0$, P_method=:purification, GF_method=:kpm, Ncheb=150, maxdim=200, cutoff=1e-8, purify_method=:mcweeny, purify_maxdim=40, purify_maxiters=30, purify_tol=1e-5, $\eta=1e-3$, krylov_nsweeps=12, krylov_maxdim=100, krylov_cutoff=1e-8, verbose=false) → MPO</code>	Non-interacting polarization bubble $\Pi_0(\omega)$ on <code>H1.sites</code> . <code>P_method=:purification</code> reuses <code>H.density_cache</code> when present; <code>P_method=:kpm</code> computes P via <code>get_density_from_Tn</code> . <code>GF_method=:kpm</code> (auto spectral-bound estimation) or <code>:krylov</code> (via <code>get_green_krylov</code>) for $G_{\text{eff}}(\omega)$, $H_{\text{eff}} = \mathbf{I} \otimes H_2 - H_1 \otimes \mathbf{I}$.
<code>get_rpa_susceptibility(H::TBHamiltonian, MPOV, ω; mode=:charge, rpa_nsweeps=20, rpa_maxdim=400, rpa_cutoff=1e-8, ⟨get_bubble_mpo kwargs⟩) → MPS</code>	RPA susceptibility $\chi(\omega)$, diagonal χ_{ijjj}^{RPA} on a $2L$ -site MPS. <code>mode=:charge</code> (default) computes the density-density bubble via <code>get_bubble_mpo(H,H,ω)</code> ; <code>mode=:magnetic</code> projects <code>H</code> onto its spin- $\uparrow/\downarrow$ blocks and calls <code>get_magnon_bubble</code> (requires <code>H.spin_s != nothing</code>). Solves $(I - \Pi_0 V)\chi = \Pi_0$ via <code>rpa_from_bubble_diag</code> .

Magnon (Transverse Spin) Susceptibility

Signature	Description
<code>get_magnon_bubble(H::TBHamiltonian, ω; ⟨get_bubble_mpo kwargs⟩) → MPO</code>	Transverse spin bubble $\Pi_0^{+-}(\omega) = \sum_k (f_{k\downarrow} - f_{k\uparrow}) / (\omega - (\epsilon_{k\uparrow} - \epsilon_{k\downarrow}) + i\eta)$. Projects <code>H</code> onto spin- $\uparrow/\downarrow$ blocks via <code>_project_spin_sector</code> and calls <code>get_bubble_mpo(H.up,H.dn,ω)</code> . Errors if <code>H.spin_s === nothing</code> .
<code>get_magnon_susceptibility(H::TBHamiltonian, MPOV, ω; rpa_nsweeps=20, rpa_maxdim=400, rpa_cutoff=1e-8, ⟨get_bubble_mpo kwargs⟩) → MPS</code>	RPA transverse spin susceptibility $\chi_{\text{RPA}}^{+-}(\omega)$. Builds $\Pi_0^{+-}(\omega)$ via <code>get_magnon_bubble</code> , then solves $(I - \Pi_0 V)\chi = \Pi_0$. For a Hubbard-like interaction, <code>MPOV = U · Id</code> .
<code>get_magnon_susceptibility_wynn(H::TBHamiltonian, MPOV, ωlist; K_max=6, maxdim_apply=200, cutoff_apply=1e-8, verbose=false, ⟨get_bubble_mpo kwargs⟩) → (chi_partial, chi_wynn)</code>	Wynn- ϵ -accelerated transverse spin RPA susceptibility over a frequency list. Spin- $\uparrow/\downarrow$ projections are done once outside the ω loop; otherwise identical to <code>get_rpa_susceptibility_wynn</code> .

Wynn ϵ -Accelerated RPA (Neumann Series)

Instead of solving $(I - \Pi_0 V)\chi = \Pi_0$ directly, builds the Neumann series $T_0 = \Pi_0$, $T_n = T_{n-1} \cdot V \cdot \Pi_0$ (so $\sum_n T_n \rightarrow \chi^{\text{RPA}}$ as $K \rightarrow \infty$), extracts k -space scalars $s_n(q, \omega) = -\text{Im}\langle q | T_n(\omega) | q \rangle$ via `get_spect_k`, and applies the Wynn ϵ -algorithm to the partial-sum sequence for fast convergence.

Signature	Description
<code>wynn_epsilon(s) → Vector{ComplexF64}</code>	Wynn ε -algorithm on a scalar sequence $s = [s_0, \dots, s_K]$. Returns even-column first-row Padé estimates $[\varepsilon_2(0), \varepsilon_4(0), \dots]$. Uses only additions and reciprocals.
<code>get_spect_k(W::MPO; tol=1e-9, maxdim=100) → Vector{ComplexF64}</code>	k -space diagonal of MPO W as a dense length- 2^L vector. Conjugates W by the QFT, extracts the diagonal as an MPS, and evaluates $\langle k \tilde{W} k \rangle$ for all 2^L quantics k -indices (LSB-first convention).
<code>get_rpa_susceptibility_wynn(H::TBHamiltonian, ωlist; mode=:charge, K_max=6, maxdim_apply=200, cutoff_apply=1e-8, verbose=false, $\langle$get_bubble_mpo kwargs$\rangle$) → (chi_partial, chi_wynn)</code>	$\chi_{\text{rpa}}(q, \omega)$ for all ω in ω list via Wynn Padé acceleration of the bubble series. mode=:charge (default) uses <code>get_bubble_mpo(H, H, ω)</code> ; mode=:magnetic uses <code>get_magnon_bubble</code> (requires <code>H.spin_s != nothing</code>). <code>chi_partial[k+1, iω, q]</code> = partial sum $\sum_{n=0}^k s_n(q, \omega)$; <code>chi_wynn[m, iω, q]</code> = $\varepsilon_{2m}(0)$ estimate.
<code>rpa_wynn_from_bubbles(Π_0_list, MPOV; K_max=6, maxdim_apply=200, cutoff_apply=1e-8, verbose=false) → (chi_partial, chi_wynn)</code>	Same as <code>get_rpa_susceptibility_wynn</code> but takes a precomputed <code>Vector{MPO}</code> of bubbles directly (e.g. from <code>get_bubble_mpo_cheb2d_tucker</code> or <code>get_bubble_mpo_haydock</code>), skipping the internal bubble calculation.

Haydock Recursion (Operator-Level Krylov)

An alternative to KPM/linsolve for $G_{\text{eff}}(\omega)$: builds an orthogonal Krylov basis $\{\Phi_n\}$ of MPOs under H_{eff} via Lanczos recursion with the Frobenius inner product $(A, B) = \text{Tr}[A^\dagger B]$, then evaluates the resolvent for any ω by solving the small tridiagonal Lanczos system — the expensive recursion runs once and is reused for every frequency.

Signature	Description
<code>haydock_cf(H_mpo, seed, N_steps; maxdim=200, cutoff=1e-8, verbose=false) → (a, b, basis, norm0)</code>	Lanczos recursion $\Phi_{n+1} = H \cdot \Phi_n - a_n \Phi_n - b_n \Phi_{n-1}$ on MPO vectors. Returns diagonal coefficients a , off-diagonal b (<code>b[1] = seed _F</code>), the normalized Krylov basis MPOs, and norm0 . Stops early at an invariant subspace.
<code>eval_haydock_cf(a, b, z) → ComplexF64</code>	Evaluate the continued fraction $\langle \text{seed} (z - H)^{-1} \text{seed} \rangle$ by backward recursion; <code>b[1]</code> must be norm0 .
<code>haydock_resolve_mpo(a, b, basis, z; maxdim=200, cutoff=1e-8) → MPO</code>	Reconstruct $(z - H)^{-1} \text{seed} \rangle$ by solving the $N \times N$ Lanczos tridiagonal system $(zI - T)c = b_1 e_1$ and forming $\Pi_0(z) = \sum_n c_n \cdot \text{basis}_n$.

<code>get_bubble_mpo_haydock(H1, H2, ωlist; N_steps=30, η=1e-2, maxdim=200, cutoff=1e-8, ϵ_F=0.0, P_method=:purification, purify_method=:mcweeny, purify_maxdim=40, purify_maxiters=30, purify_tol=1e-5, Ncheb=150, verbose=false) → Vector{MPO}</code>	Bare polarization bubble $\Pi_0(\omega)$ as an L -site MPO for each ω in ω list. Builds the Krylov basis once from seed $\Phi_0 = \mathbf{I} \otimes P_2 - P_1 \otimes \mathbf{I}$ on $H_{\text{eff}} = \mathbf{I} \otimes H_2 - H_1 \otimes \mathbf{I}$; for each ω the resolvent $(\omega + i\eta - H_{\text{eff}})^{-1}\Phi_0$ is recovered from the stored Lanczos data. Output compatible with <code>rpa_wynn_from_bubbles</code> .
---	--

Double Chebyshev (2D) Bubble Decomposition

Alternative to running KPM directly on the $2L$ -site H_{eff} (where bond dimension grows with each Chebyshev step due to inter-subsystem entanglement). Instead, $G_{\text{eff}}(\omega)$ is decomposed as $G_{\text{eff}}(\omega) \approx \sum_{mn} c_{mn}(\omega) T_m(\tilde{H}_1) \otimes T_n(\tilde{H}_2)$, where T_m, T_n are Chebyshev polynomials of the L -site rescaled Hamiltonians and $c_{mn}(\omega)$ are cheap scalar 2D Chebyshev coefficients (via 2D DCT-II). The bubble is then $\Pi_0(\omega) = \sum_{mn} c_{mn}(\omega) D_{mn}$ with $D_{mn} = (T_m(\tilde{H}_1)P_1) \odot T_n(\tilde{H}_2) - T_m(\tilde{H}_1) \odot (T_n(\tilde{H}_2)P_2)$ (Hadamard product $\odot$ via `hadamard_mpo`). Plain, Tucker-compressed, and per- ω -SVD variants trade off precomputation cost against the number of Hadamard/QFT operations; all share the keyword arguments listed for `get_bubble_mpo_cheb2d` unless noted otherwise.

Signature	Description
<code>chebyshev2d_gf_coeffs(ω, scale1, center1, scale2, center2, η, N) → Matrix{ComplexF64}</code>	2D Chebyshev expansion coefficients c_{mn} of $f(x, y) = 1/(\omega + i\eta - (\text{scale2} \cdot y + \text{center2} - \text{scale1} \cdot x - \text{center1}))$ on $[-1, 1]^2$ via an $N \times N$ Chebyshev-Gauss grid and 2D DCT-II. N should equal <code>length(Tn_list)</code> from <code>KPM_Tn</code> (i.e. <code>Ncheb+1</code>).
<code>get_bubble_mpo_cheb2d(H1, H2, ωlist; Ncheb=50, maxdim=200, cutoff=1e-8, ϵ_F=0.0, P_method=:purification, purify_method=:mcweeny, purify_maxdim=40, purify_maxiters=30, purify_tol=1e-5, η=1e-3, coeff_tol=1e-12, verbose=false) → Vector{MPO}</code>	Full-MPO bubble $\Pi_0(\omega)$ for each ω via the plain double Chebyshev decomposition. All $C[m, n](\omega)$ are precomputed once; the (m, n) double loop runs once, each D_{mn} is computed once and accumulated into every $\Pi(\omega)$. <code>coeff_tol</code> prunes negligible (m, n) pairs/rows.
<code>get_bubble_mpo_cheb2d.tucker(H1, H2, ωlist; (get_bubble_mpo_cheb2d kwargs), tucker_tol=1e-3, tucker_maxrank=20, kernel=:jackson, hooi_iters=3) → Vector{MPO}</code>	Tucker-accelerated variant. HOSVD+HOOI finds global bases U_m ($N \times r_m$) and V_n ($N \times r_n$) with $C[m, n](\omega) \approx U_m G(\omega) V_n^\dagger$. Computes $r_m + r_n$ weighted MPO sums and $r_m \times r_n$ Hadamard products once; per- ω cost is $r_m \times r_n$ scalar-weighted MPO additions. Speedup: $N^2 \rightarrow r_m r_n$ Hadamard products. <code>kernel=:jackson</code> applies <code>_jackson_kernel</code> damping to reduce Tucker rank.
<code>get_bubble_diag_cheb2d(H1, H2, ωlist; (get_bubble_mpo_cheb2d kwargs), qft_tol=1e-9, qft_maxdim=100) → Vector{MPS}</code>	Diagonal-only variant of <code>get_bubble_mpo_cheb2d</code> . Returns $\text{diag } \Pi_0(k, \omega) = \langle k \Pi_0(\omega) k \rangle$ as one MPS per ω . Each D_{mn} is QFT-conjugated and diagonal-extracted once; $c_{mn}(\omega) \cdot \text{diag}(D_{mn})$ is accumulated as cheap MPS sums.

<code>get_bubble_diag_cheb2d_svd(H1, H2, ωlist; $\langle$get_bubble_diag_cheb2d kwargs$\rangle$, svd_tol=1e-6, svd_maxrank=20, kernel=:jackson) $\rightarrow$ Vector{MPS}</code>	Per- ω SVD-accelerated variant of <code>get_bubble_diag_cheb2d</code> . Each $C[m, n](\omega)$ is rank-truncated by its own SVD ($C(\omega) = \sum_s \sigma_s u_s v_s^\dagger$, typically rank 2–5 with the Jackson kernel); one Hadamard product and one QFT per (ω, s) .
<code>get_bubble_diag_cheb2d_tucker(H1, H2, ωlist; $\langle$get_bubble_diag_cheb2d kwargs$\rangle$, tucker_tol=1e-3, tucker_maxrank=20, kernel=:jackson, hooi_iters=3) $\rightarrow$ Vector{MPS}</code>	Tucker (HOSVD+HOOI) variant of <code>get_bubble_diag_cheb2d</code> . Global bases U_m, V_n shared across all ω ; $r_m \times r_n$ Hadamard+QFT operations total, then per- ω cost is scalar-weighted MPS additions over the core tensor $\tilde{A}[s_1, s_2, \omega] = U_m^\top C(\omega) V_n$.

Low-Level Bubble Pipeline

Legacy 4-step pipeline retained as building blocks (the high-level `get_bubble_mpo`/`get_rpa_susceptibility` are preferred for new code):

1. `get_Tnlists` — build Chebyshev lists for system 1, system 2, and the effective combined Hamiltonian $H_{\text{eff}} = \mathbf{I} \otimes H_2 - H_1 \otimes \mathbf{I}$.
2. `get_bubble_expanded_from_Tn` — compute $\Pi_0(\omega)$ as a $2L$ -site MPO.
3. `build_bubble_mpo` — collapse to an L -site MPO.
4. `rpa_from_bubble_diag` — solve the Dyson equation via DMRG linear solver.

Signature	Description
<code>get_Tnlists(H, H2, sites, sites2, N; a=6, maxdim=100) $\rightarrow$ (Tn1, Tn2, Tn_eff)</code>	Build 3 Chebyshev lists: individual and effective.
<code>get_bubble_expanded_from_Tn(Tn1, Tn2, Tn_eff, s1, s2, N, ω, ε_F; a=6, maxdim=200) $\rightarrow$ MPO</code>	Lindhard bubble $\Pi_0(\omega)$ as a $2L$ -site MPO.
<code>build_bubble_mpo(ω; Tn1, Tn2, Tn_eff, sites, sites2, finalsites, finalfinalsites, ...) $\rightarrow$ MPO</code>	Collapse $2L$ -site bubble to L -site MPO.
<code>rpa_from_bubble_diag(Π, V, finalsites, finalfinalsites; nsweeps=20, maxdim=400, cutoff=1e-8) $\rightarrow$ MPS</code>	Solve $(I - \Pi_0 V)\chi = \Pi_0$ via DMRG linear solver.

Tensor Utilities (used internally by RPA)

Signature	Description
<code>mpo_kron(A, B) $\rightarrow$ MPO</code>	Concatenate two MPOs (Kronecker product on site space).
<code>mps_kron(A, B) $\rightarrow$ MPS</code>	Concatenate two MPS.
<code>replace_sites(MPO, newsites) $\rightarrow$ MPO</code>	Remap physical indices of an MPO to <code>newsites</code> , preserving prime levels.

<code>interleave_mpo(mpo, phys_sites, n) → MPO</code>	Embed L -site MPO into $2L$ -site space by interleaving with identity. <code>n=0</code> : operator at even positions; <code>n=1</code> : operator at odd positions.
<code>interleave_mpo_tb(op, sites_A, sites_B, which) → MPO</code>	Generalization of <code>interleave_mpo</code> for heterogeneous site spaces (Layer, Honeycomb, ...). Embeds <code>op</code> into the $2N$ -site product space ordered <code>[sites_A[1], sites_B[1], ...]</code> . <code>which=:A</code> or <code>:B</code> selects which site set <code>op</code> acts on.
<code>swap_every_other_legs(MPOin, newsites) → MPO</code>	Replace site indices and swap bra $\leftrightarrow$ ket on every even-numbered site; converts a $2L$ -site interleaved bubble MPO into the form expected by <code>collapse_mpo_pairs</code> .
<code>collapse_mpo_pairs(mpo2L, out_sites) → MPO</code>	Merge consecutive site pairs $(2n-1, 2n)$ to single site: $2L \rightarrow L$.
<code>extract_diagonal_to_mps(M) → MPS</code>	Extract diagonal of an MPO as an MPS by projecting bra=ket.
<code>merge_mps_to_mpo(mps) → MPO</code>	Contract consecutive pairs of an MPS into MPO tensors.
<code>convert_mpo(old_mps, new_sites) → MPO</code>	Convert a $2N$ -site MPS (typically from QTCl) into an N -site MPO by merging pairs via <code>merge_mps_to_mpo</code> and remapping physical indices to <code>new_sites</code> .
<code>apply_interleave_swaps(W, sites; cutoff=1e-16, maxdim=200, verbose=false) → MPO</code>	Reorder sites from block order $[1 \dots N, N+1 \dots 2N]$ to interleaved via SWAP gates.

Self-Consistent Mean Field (SCF_tk.jl)

Generic self-consistent mean-field (Hartree–Fock-like) loops for `TBHamiltonian`, built around a density $\rightarrow$ mean-field $\rightarrow$ density fixed-point iteration. The physical channel (CDW, magnetism, s -/ p -wave pairing) is encoded entirely in user-supplied `hartree_builder`/`fock_builder` closures of the form `builder(density, sites) -> MPO`, so all channels share the same generic driver `scf_meanfield`. The density matrix at each iteration comes from `get_density (density_method=:sp2|:mcwee`

Generic Driver

Signature	Description
-----------	-------------

```
scf_meanfield(H0::TBHamiltonian,
hartree_builder;
initial_density=nothing,
initial_hartree=nothing,
fock_builder=nothing,
initial_fock=nothing,
density_method=:sp2,
density_mode=:direct, Nel=H0.N÷2,
fermi=0.0, Ncheb=100, scale=nothing,
spectral_bounds=nothing,
purification_scale_padding=1.05,
max_scf_iter=30, scf_tol=1e-6,
mix=0.4, maxdim=100, cutoff=1e-8,
purif_maxiter=40, purif_tol=1e-6,
stop_on_increase=false, verbose=true)
→ NamedTuple
```

Generic mean-field loop. Each iteration: (1) build $H_{\text{MF}} = H_0 + V_{\text{Hartree}}(+V_{\text{Fock}})$; (2) compute the density matrix via `get_density`; (3) extract a density profile (`density_mode=:direct` or `:complement`); (4) compute the RMS error against the previous profile, linearly mix with weight `mix`, and rebuild the Hartree/Fock terms via the supplied builders. `stop_on_increase=true` returns the best-so-far state early if the residual increases. Returns (`converged`, `stopped_by_residual_increase`, `iterations`, `history`, `rms_error`, `density_mpo`, `density_mps`, `hartree_mpo`, `fock_mpo`, `ham`), with `history` a `Vector{NamedTuple}` of per-iteration diagnostics.

Channel-Specific Drivers

Signature	Description
<pre>scf_magnetic_hubbard(H0::TBHamiltonian, U::Union{Number,MPO}; initial_up=nothing, initial_dn=nothing, background=0.5, density_method=:sp2, Nel_up=H0.N÷2, Nel_dn=H0.N÷2, fermi=0.0, Ncheb=100, scale=nothing, purification_scale_padding=1.05, max_scf_iter=30, scf_tol=1e-6, mix=0.4, maxdim=100, cutoff=1e-8, purif_maxiter=40, purif_tol=1e-6, verbose=true) → NamedTuple</pre>	Two-channel collinear magnetic (Hubbard) mean field: $H_{\uparrow} = H_{0\uparrow} + U(n_{\downarrow} - \text{background})$, $H_{\downarrow} = H_{0\downarrow} + U(n_{\uparrow} - \text{background})$. If H_0 is spinful, $H_{0\uparrow}/H_{0\downarrow}$ are obtained by projecting out the spin core; if spinless both channels share H_0. $U::\text{MPO}$ gives a non-local Hartree kernel via <code>hartree_mpo_from_density</code>. Default initial guess is the staggered (Néel) profile from <code>staggered_magnetic_initial</code>. Returns (<code>converged</code>, <code>iterations</code>, <code>rms_error</code>, <code>rho_up</code>, <code>rho_dn</code>, <code>density_up_mpo</code>, <code>density_dn_mpo</code>, <code>H_up</code>, <code>H_dn</code>, <code>history</code>).

<pre>scf_swave_superconducting(H0::TBHamiltonian, g::Number; initial_delta=0.1, initial_up=nothing, initial_dn=nothing, pairing_sign=-1.0, include_hartree=false, hartree_coupling=g, hartree_sign=-1.0, background=0.5, mu=0.0, density_method=:mcweeny, normal_density_mode=:particle, Ncheb=100, scale=nothing, purification_scale_padding=1.05, max_scf_iter=30, scf_tol=1e-6, mix=0.4, maxdim=100, cutoff=1e-8, purif_maxiter=40, purif_tol=1e-6, verbose=true) → NamedTuple</pre>	Spin-singlet s-wave BdG pairing SCF: $\Delta_{\text{new}}(i) = \text{pairing_sign} \cdot g \cdot F_{\text{singlet}}(i)$, with F_{singlet} extracted from the Nambu off-diagonal block of the BdG density matrix via <code>swave_anomalous_profile</code>. <code>include_hartree=true</code> additionally self-consistently updates spin-resolved onsite Hartree potentials $V_{\uparrow/\downarrow}(i) = \text{hartree_sign} \cdot \text{hartree_coupling} \cdot (n_{\downarrow/\uparrow}(i) - \text{background})$ via <code>swave_normal_profiles</code>. If H0 is spinless, a spin degree of freedom is added to a working copy via <code>add_spin!</code> before building the BdG Hamiltonian. Returns (<code>converged</code>, <code>iterations</code>, <code>rms_error</code>, <code>delta_mps</code>, <code>anomalous_mps</code>, <code>rho_up_mps</code>, <code>rho_dn_mps</code>, <code>hartree_up_mpo</code>, <code>hartree_dn_mpo</code>, <code>density_mpo</code>, <code>ham</code>, <code>history</code>).
<pre>scf_swave_hubbard(H0::TBHamiltonian, U::Number; pairing_sign=-1.0, hartree_coupling=U, hartree_sign=-1.0, kwargs...) → NamedTuple</pre>	Attractive on-site Hubbard s-wave SCF; thin wrapper around <code>scf_swave_superconducting</code> with <code>include_hartree=true</code>. Positive U gives attraction: $\Delta_i = -UF_i$, $V_{\uparrow/\downarrow} = -U(n_{\downarrow/\uparrow} - \text{background})$.
<pre>scf_pwave_equalspin(H0::TBHamiltonian, V::Number; initial_up=0.1, initial_dn=nothing, distance=1, eta_down=-1.0, pairing_sign=-1.0, mu=0.0, density_method=:mcweeny, Ncheb=100, scale=nothing, purification_scale_padding=1.05, max_scf_iter=30, scf_tol=1e-6, mix=0.4, maxdim=100, cutoff=1e-8, purif_maxiter=40, purif_tol=1e-6, verbose=true) → NamedTuple</pre>	Minimal spinful equal-spin p-wave triplet SCF on nearest-neighbor (<code>distance</code>) bonds: $\Delta_{\uparrow/\downarrow}(i) = \text{pairing_sign} \cdot V \cdot F_{\uparrow/\downarrow}(i, i + \text{distance})$, with $F_{\uparrow/\downarrow}$ from <code>pwave_equalspin_anomalous_profiles</code>. <code>eta_down=-1</code> (default) seeds a helical configuration ($\Delta_{\downarrow}(i) = -\Delta_{\uparrow}(i)$ initially); <code>eta_down=1</code> starts both channels with the same sign. Returns (<code>converged</code>, <code>iterations</code>, <code>rms_error</code>, <code>delta_up_mps</code>, <code>delta_dn_mps</code>, <code>anomalous_up_mps</code>, <code>anomalous_dn_mps</code>, <code>density_mpo</code>, <code>ham</code>, <code>history</code>).

Hartree/Fock Builder Factories

Each factory returns a closure `builder(density, sites) -> MPO` suitable for `hartree_builder/fock_builder` in `scf_meanfield`, or for the `U/interaction` arguments of `get_scf`.

Signature	Description
<pre>hartree_mpo_from_density(density_mps::MPS, interaction_op::MPO, sites; background=nothing, maxdim=100, cutoff=1e-8) → MPO</pre>	Apply interaction kernel <code>interaction_op</code> to $(\rho - \text{background})$ and convert the resulting coefficient MPS to a diagonal Hartree MPO via <code>mps_to_diagonal_mpo</code>. <code>background</code> may be <code>nothing</code>, a <code>Number</code>, or an <code>MPS</code>.
<pre>fock_mpo_from_density(density_mpo::MPO, interaction_op::MPO, sites; sign=-1, maxdim=100, cutoff=1e-8) → MPO</pre>	Fock/exchange MPO $F_{ij} = \text{sign} \cdot V_{ij} \cdot \rho_{ij}$ via <code>hadamard_mpo</code>.

<code>fock_exchange_builder(interaction_op::MPO, sign=-1, maxdim=100, cutoff=1e-8) → Function</code>	Wrap <code>fock_mpo_from_density</code> as a <code>fock_builder(density_mpo, sites)</code> closure for <code>scf_meanfield</code> .
<code>cdw_hartree_builder(U::Number; background=0.5, maxdim=100, cutoff=1e-8) → Function</code>	Local spinless CDW Hartree builder: $V_H(n) = U(\rho(n) - \text{background})$.
<code>dense_hartree_builder(V, L, sites; background=nothing, type=Float64, tol=1e-8, maxdim=100, cutoff=1e-8, kwargs...) → Function</code>	Hartree builder for a dense interaction kernel. V is either an existing MPO or a function $V(i, j)$ of 0-indexed coordinates, in which case it is QTCI-compressed via <code>get_mpo</code> .
<code>pair_distance_interaction_mpo(L, sites, distance::Integer, weight=1; type=Float64, tol=1e-8, maxdim=100, cutoff=1e-8) → MPO</code>	Sparse interaction MPO for fixed-distance site pairs $(i, i+d)$, built from non-cyclic <code>shift_pair_mpos</code> . <code>weight</code> may be a Number, Function $w(i)$, or MPO. The vector form sums multiple <code>distance => weight</code> (or <code>(distance, weight)</code>) terms.
<code>pair_distance_interaction_mpo(L, sites, terms::AbstractVector; kwargs...) → MPO</code>	
<code>pair_distance_hartree_builder(L, sites, distance_or_terms, weight=1; background=nothing, type=Float64, tol=1e-8, maxdim=100, cutoff=1e-8) → Function</code>	Hartree builder wrapping <code>pair_distance_interaction_mpo</code> via <code>hartree_mpo_from_density</code> .

Unified Entry Points

Signature	Description
<code>get_scf(H0::TBHamiltonian, channel::Symbol; interaction=:dense, kwargs...) → NamedTuple</code>	Reads the interaction from <code>H0.interaction_mpo</code> (set via <code>add_interaction!</code>) and dispatches to the 3-argument <code>get_scf(H0, U, channel)</code> . Supported: <code>:cdw</code> , <code>:magnetic</code> . <code>:swave</code> / <code>:pwave</code> raise an informative error since they require an explicit coupling constant.

```

get_scf(H0::TBHamiltonian, U,
channel::Symbol; interaction=:local,
background=0.5, distance=1,
initial_density=nothing,
initial_hartree=nothing,
density_mode=:direct,
density_method=:sp2, method=nothing,
Nel=H0.N÷2, fermi=0.0, Ncheb=100,
scale=nothing,
spectral_bounds=nothing,
purification_scale_padding=1.05,
tol=1e-6, maxiters=30, mixing=0.4,
maxdim=100, cutoff=1e-8,
purif_maxiter=40, purif_tol=1e-6,
stop_on_increase=false, verbose=true,
builder_kwargs...) → NamedTuple

```

Convenience front-end dispatching on `channel` (canonicalized via `_canonical_channel`, with aliases). `:CDW` (aliases `:charge`, `:hartree`): builds a Hartree builder according to `interaction` in `{:local, :dense, :distance}` — `:local` requires numeric `U` (`cdw_hartree_builder`), `:dense` uses `dense_hartree_builder`, `:distance/:pair(s)` uses `pair_distance_hartree_builder` (vector `U` for multiple terms) — then calls `scf_meanfield`. `:magnetic` (aliases `:magnetism`, `:spin`, `:hubbard`) calls `scf_magnetic_hubbard`. `:swave` (aliases `:s_wave`, `:superconducting`, ...) requires numeric `U` and calls `scf_swave_hubbard`. `:pwave` (aliases `:p_wave`, `:triplet`) requires numeric `U` and calls `scf_pwave_equalspin`. `method/density_method` accept aliases (`:purification`, `:sp2` → `:sp2`; `:mcweeny` → `:mcweeny`; `:kpm`, `:chebyshev` → `:kpm`) via `_canonical_density_method`.

Density and Pairing Profile Extraction

Signature	Description
<code>density_profile_from_dm(density_mpo::MPO, sites; mode=:direct) → MPS</code>	Diagonal of a density-matrix MPO as an MPS. <code>mode=:complement</code> returns $1 - \text{diag}(D)$.
<code>staggered_magnetic_initial(H::TBHamiltonian, amplitude=0.05, background=0.5) → (rho_up, rho_dn)</code>	Antiferromagnetic initial guess for <code>scf_magnetic_hubbard</code> : $\rho_{\uparrow/\downarrow}(n) = \text{background} \pm \text{amplitude} \cdot (-1)^n$. If <code>H</code> is spinful, the spin core is projected out first.
<code>swave_anomalous_profile(density_mpo::MPO, Hbdg::TBHamiltonian; maxdim=100, cutoff=1e-8) → MPS</code>	Local singlet anomalous profile from the Nambu particle-hole block of a spinful <i>s</i> -wave BdG density matrix, projected onto the spin-singlet combination $\frac{1}{2}(f_{\uparrow\downarrow} - f_{\downarrow\uparrow})$. Requires <code>Hbdg.nambu_s</code> and <code>Hbdg.spin_s</code> to be set.
<code>swave_normal_profiles(density_mpo::MPO, Hbdg::TBHamiltonian; mode=:particle, maxdim=100, cutoff=1e-8) → (rho_up, rho_dn)</code>	Spin-resolved local normal densities from a spinful BdG density matrix. <code>mode=:particle</code> reads the particle-particle Nambu block directly; <code>mode=:hole_complement</code> returns $1 - \text{diag}(\text{hole-hole block})$.

`pwave_equalspin_anomalous_profiles(density_mpo, neighbor (distance) equal-spin
Hbdg::TBHamiltonian; distance=1, triplet anomalous bond profiles from the
maxdim=100, cutoff=1e-8) → (F_up, Nambu off-diagonal block of a spinful BdG
F_dn) density matrix; the profile value at i is the
bond $(i, i+distance)$.`

Néel/Trivial Initial-Guess Density Matrices

Seeds for mean-field SCF on interacting models; each returns a diagonal density (MPO, MPS) pair built via QTCI of an explicit occupation function.

Signature	Description
<code>initial_guess_trivial_up_1D(L, sites) → (MPO, MPS)</code>	1D spin-up Néel seed: occupation $x \bmod 2$ on site x .
<code>initial_guess_trivial_down_1D(L, sites) → (MPO, MPS)</code>	1D spin-down Néel seed: occupation $(x+1) \bmod 2$ on site x .
<code>initial_guess_Neel_up(Lx, Ly, sites) → (MPO, MPS)</code>	2D checkerboard spin-up seed: occupation 1 where $i_x + i_y$ is even.
<code>initial_guess_Neel_dn(Lx, Ly, sites) → (MPO, MPS)</code>	2D checkerboard spin-down seed: occupation 1 where $i_x + i_y$ is odd.

Utilities (utils.jl)

Binary Encoding and MPS/MPO Conversions

Signature	Description
<code>to_binary_vector(n, L) → Vector{String}</code>	Convert integer n to length- L big-endian binary string vector.
<code>binary_to_MPS(n, L, sites) → MPS</code>	Computational-basis state $ n\rangle$ as an L -site MPS.
<code>matrix_checker(mpo, sites, i, j) → Number</code>	Matrix element $\langle i \text{mpo} j \rangle$ by contracting basis MPS (small systems only). Optional L argument accepted but ignored.
<code>get_matrix(mpo, sites) → Matrix</code>	Full $D \times D$ dense matrix of <code>mpo</code> (feasible for $D \lesssim 512$). Optional L argument accepted but ignored. Works for any site types.
<code>fix_sites(mpo, sites) → MPO</code>	Replace site indices of an MPO (e.g. from TCI) with target <code>sites</code> .
<code>custom_mpo(mps, new_sites) → MPO</code>	Convert a $2N$ -site MPS (interleaved QTCI) into an N -site MPO.
<code>fused_mpo(mps, new_sites) → MPO</code>	Convert an N -site MPS with dim-4 physical indices (fused grid) into an N -site MPO.
<code>mps2mpo(L, sites, density_mps) → MPO</code>	Promote a diagonal MPS to a diagonal MPO via <code>_asdiagonal</code> .
<code>custom_mps(qtt, sites) → MPS</code>	Replace site indices of a TCI MPS with target <code>sites</code> .
<code>build_shift_mpo(sites, q) → MPO</code>	Cyclic shift MPO: maps $ x\rangle \rightarrow x+q\rangle \pmod{2^L}$ on a binary-encoded chain. Built site-by-site using binary addition with carry; q is an integer offset.

<code>mpsexciton(x, sites) → MPS</code>	Exciton basis state $ x, x\rangle$ on the interleaved $2L$ -site electron-hole chain. <code>x</code> is 1-indexed; odd sites = electron qubit, even sites = hole qubit.
<code>mps_to_diagonal_mpo(mps, sites) → MPO</code>	Convert an MPS to a diagonal MPO on <code>sites</code> by replacing each physical index with a bra-ket pair tied by a delta tensor. Used to embed QTCI-compressed 2D functions (e.g. from <code>build_interaction_op_exciton</code>) into MPO form.

Profile Evaluation and Spatial Sampling

`eval_mps` evaluates a profile MPS pointwise; `spatial_sampling_plan` is the shared geometry-aware sampler — used by `eval_mps_spatial` below and by `get_ldos_spatial` and its GPU counterparts — that turns `num_x/num_y/ grid/reduce/sublattice` kwargs into a list of sample positions.

Signature	Description
<code>eval_mps(A::MPS, n::Int) → Real</code>	Evaluate a profile MPS <code>A</code> at the 0-indexed basis coordinate <code>n</code> by contracting with the basis state $ n\rangle$, using the big-endian (MSB-first) convention of <code>binary_to_mps</code> . Not the same convention as <code>_eval_diag_mps</code> , which is LSB-first (QFT momentum ordering); mixing the two bit-reverses the site map.
<code>spatial_sampling_plan(L; Lx=nothing, grid=false, reduce=:point, n_sub=1, num_x=0, num_y=nothing, num_avg=1, x_start=1, x_end=2^L, xwin=nothing, ywin=nothing, x_groups=nothing, box_half=0, sublattice=:auto) → NamedTuple</code>	Geometry-aware real-space sampling plan shared by every spatial sampler. Returns <code>(centers, groups, resolve_sublattice, n_sub, stride_x, stride_y, grid, reduce, a, b)</code> . <code>reduce=:point</code> (default) lays out <code>num_x[×num_y]</code> sample positions/groups, optionally averaged over a <code>box_half</code> neighbourhood — cheap, but can alias features narrower than the sampling grid. <code>reduce=:block</code> partitions the system into <code>num_x×num_y</code> power-of-two blocks ($a = \log_2 \text{num}_x$, $b = \log_2 \text{num}_y$; <code>centers</code> mark block midpoints) whose values are obtained by tracing out the within-block bits (see <code>_eval_block_mps</code>) — gap-free and independent of block size, so it scales to $L_x, L_y \gtrsim 14$. For multi-atom unit cells (<code>n_sub>1</code>), <code>sublattice</code> chooses whether to <code>:resolve</code> (one output column per atom), <code>:average</code> (trace out the sublattice), or decide automatically (<code>:auto</code> : resolve only for atomic-scale <code>:point</code> sampling with unit stride and no box averaging; average otherwise). <code>grid=true</code> (needs <code>Lx</code>) lays centers on a 2D <code>num_x×num_y</code> grid over the 0-indexed window <code>xwin/ywin</code> ; otherwise centers follow a 1D linear sweep of <code>x_start...x_end</code> , or the explicit <code>x_groups</code> .

```
eval_mps_spatial(A::MPS;
  num_x=2^L, num_avg=1,
  x_start=1, x_end=2^L,
  x_groups=nothing, box_half=0,
  Lx=nothing) → (values,
  centers, groups)
```

Spatial sampler for a profile MPS such as an SCF density profile (e.g. `res.rho.up`). Builds a sampling plan via `spatial_sampling_plan` and returns, for each group, the mean of `eval_mps(A, x-1)` over the group's (1-indexed) site indices — the same `num_x/num_avg/x_groups/box_half` API as `get_ldos_spatial`, but using direct MPS evaluation instead of a KPM recursion, so sweeping a large system by sampling-and-averaging is cheap. `Lx` (default $L \div 2$) sets the 2D layout used when `box_half > 0`.

`_eval_block_mps(A, ixp, iyp, a, b, Lx, Ly)` is the internal helper behind `reduce=:block`: it returns the sum of `A` over one coarse block by tracing out the within-block position bits (contracted with `[1,1]`) and pinning the kept high-order block bits to the coarse pixel (`ixp, iyp`), on the big-endian site order `[iy_MSB..iy_LSB, ix_MSB..ix_LSB]`.

QTCI Compression and MPS/MPO Builders

Signature	Description
<code>get_mps(L, sites, f; type=Float64, tol=1e-8) → MPS</code>	Compress a scalar profile $f(n)$, $n = 0, \dots, 2^L - 1$, into an MPS via QTCI ($L=1$ is built directly as a 2-element tensor).
<code>get_mpo(L, sites, f; type=Float64, tol=1e-8, kwargs...) → MPO</code>	Compress a function into an MPO via QTCI using 0-indexed coordinates $n = 0, \dots, 2^L - 1$. Dispatches on arity: a 1-argument $f(n)$ builds a diagonal MPO via <code>get_mps + mps2mpo</code> ; a 2-argument $f(i, j)$ builds a full interaction MPO from a 2D QTCI of the interleaved legs via <code>custom_mpo</code> .
<code>constant_mps(sites, value) → MPS</code>	Rank-1 MPS with amplitude <code>value</code> on every computational-basis state, via <code>extract_diagonal_to_mps</code> .
<code>rms_error(a, b) → Float64</code>	RMS distance $\sqrt{\langle (a-b) (a-b) \rangle / 2^L}$ between two MPS over all computational-basis states; used as the SCF convergence metric (the <code>rms_error</code> field returned by <code>scf_meanfield</code>).

Shift and Hadamard Operators

Signature	Description
<code>shift_mpo(sites, q; cyclic=false) → MPO</code>	Translation operator $(Tf)(x) = f(x + q)$ on the binary-encoded chain. For $q \geq 0$ built directly via <code>build_shift_mpo</code> ; for $q < 0$ returns the adjoint of <code>build_shift_mpo(sites, -q, cyclic)</code> .
<code>shift_pair_mpos(sites, q; cyclic=false) → (MPO, MPO)</code>	$(K, K^\dagger)$ where $K = \text{shift_mpo}(\text{sites}, q; \text{cyclic})$ — the shift operator and its adjoint, as used by <code>pair_distance_interaction_mpo</code> .

<code>shift_hopping_mpo(hopping, sites, q; cyclic=false, maxdim=typemax(Int), cutoff=1e-12, apply_kwargs=NamedTuple()) → MPO</code>	Symmetrized shifted-hopping term $K \cdot \text{hopping} + K^\dagger \cdot \text{hopping}^\dagger$ for a bond at displacement q , via <code>shift_pair_mpos</code> .
<code>hadamard_mpo(A, B, out_sites; maxdim=100, cutoff=1e-8) → MPO</code>	Site-wise (Hadamard) product $C_{ij} = A_{ij}B_{ij}$ of two MPOs with matching site count, remapped onto <code>out_sites</code> . Used by <code>fock_mpo_from_density</code> to form $F_{ij} = \text{sign} \cdot V_{ij}\rho_{ij}$.

Custom Operator Extensions

Defined in `utils.jl` and `2D_lattice.jl`:

Name	Site type	Matrix
<code>sigma_plus</code>	Qubit	$\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ — binary increment (up-shift)
<code>sigma_minus</code>	Qubit	$\begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}$ — binary decrement (down-shift)
<code>sigma_d</code>	Qubit	$\begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$ — projector onto $ 1\rangle$
<code>sigma_u</code>	Qubit	$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$ — projector onto $ 0\rangle$

Built-in spin- $\frac{1}{2}$ operators (used by `prepend_spin/postpend_spin`):

Symbol	Matrix	Symbol	Matrix
<code>:Id</code>	$\mathbf{I}_2$	<code>:Sz</code>	$\frac{1}{2}\sigma_z = \text{diag}(\frac{1}{2}, -\frac{1}{2})$
<code>:Pup</code>	$\text{diag}(1, 0)$	<code>:Sp</code>	$ \uparrow\rangle\langle\downarrow $
<code>:Pdn</code>	$\text{diag}(0, 1)$	<code>:Sm</code>	$ \downarrow\rangle\langle\uparrow $
<code>:Sx</code>	$\frac{1}{2}\sigma_x$	<code>:Sy</code>	$\frac{1}{2}\sigma_y$
<code>:iSy</code>	$i\sigma_y = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$	<code>:miSy</code>	$-i\sigma_y$

Built-in Nambu operators (used by `prepend_nambu/postpend_nambu`):

<code>:Id</code>	$\mathbf{I}_2$	<code>:tz</code>	$\tau_z = \text{diag}(1, -1)$
<code>:Pp</code>	$\text{diag}(1, 0)$	<code>:tp</code>	$ p\rangle\langle h $
<code>:Ph</code>	$\text{diag}(0, 1)$	<code>:tm</code>	$ h\rangle\langle p $
<code>:tx</code>	σ_x	<code>:ty</code>	σ_y

Quick-Reference Workflows

KPM Density of States and Band Structure

```
# 1. Build Hamiltonian (scale=0 means lazy DMRG estimation)
H = get_Hamiltonian("square_2d", 1.0; L=8)
```

```
# 2. Build Chebyshev list (caches in H._tn_cache / H.scale / H.center)
Tn, scale, center = KPM_Tn(H, 200; maxdim=100)
```

```
# 3. Sweep in energy (always rescale before calling KPM functions)
omega_r = (omega_phys - center) / scale    # must be in (-1, 1)

# 4. Observables
A = get_ldos_w_from_Tn(Tn, 200, omega_r; maxdim=100) # delta(omega-H)
dos  = real(tr(A))                                # density of states
G    = get_Green_retarded_from_Tn(Tn, 200, omega_r; eta=0.02)
rho  = get_density_from_Tn(Tn, 200; fermi=omega_r_fermi)

# 5. Band structure A(k,omega) -- works for plain, spinful, BdG, layered H
Akw = get_bands(H, omega_phys_vals)          # trace over aux DOF
Akw = get_bands(H, omega_phys_vals; aux_proj=[1]) # spin-up only
```

Density Matrix Purification

```
# McWeeny: one-liner (initial guess built and cached internally)
rho_mc = mcweeny_purify(H; maxiters=30, maxdim=200, tol=1e-5, verbose=true)

# SP2: one-liner with explicit electron count
rho_sp = sp2_purify(H; Nel=H.N div 2, maxiters=40, maxdim=200)

# Low-level form (for non-standard center or external MPO):
rho0 = purification_initial_guess(H; maxdim=100, cutoff=1e-8)
rho  = mcweeny_purify(rho0; maxiters=30, maxdim=200)
```

Topological Invariants

```
# Define coordinate functions (0-indexed site i, 0-indexed length L_chain)
xfunc(i, L) = Float64(i % Int(sqrt(L)))    # x-coordinate on square lattice
yfunc(i, L) = Float64(i div Int(sqrt(L)))  # y-coordinate
sz_func(i)  = isodd(i) ? 1.0 : -1.0        # sublattice / sigma_z (1-indexed)

# 1D winding number (returns a closure; pre-computes 2 MPOs once)
winding = get_W(H, xfunc, sz_func; quenched=true, Lambda=10.0, maxdim=40)
nu = sum(real(winding(alpha)) for alpha in 1:H.N) / H.N

# 2D Chern marker (returns a closure; pre-computes C1--C4 MPOs once)
chern = get_C(H, xfunc, yfunc; quenched=true, Lambda=10.0, maxdim=500)
C_local = real(chern(H.N div 2))           # evaluate at central site
C_avg   = sum(real(chern(a)) for a in 1:H.N) / H.N
```

Bilayer and Twisted Multilayer

```
# Bernal bilayer graphene (exact interlayer, no TCI)
H_bg = bilayer_hamiltonian(:honeycomb, 4, 3;
    stacking=:Bernal, t_intra=1.0, t_inter=0.3)
# H_bg.sites = [layer_s, pos_qubits...]

# Twisted bilayer at 5 degrees
H_tbg = twisted_bilayer_hamiltonian(:triangular, 4, 4, 5.0;
    t_intra=1.0, t_inter=0.3, alpha_decay=1/16)
```

Spinful Superconductor (BdG)

```
H = get_Hamiltonian("square_2d", 1.0; L=8)
add_onsite!(H, _ -> -2.0)           # chemical potential
add_zeeman!(H, 0.1; direction=:z)   # spin-splitting
add_soc!(H, 0.05; type=:rashba)     # Rashba SOC
add_superconductivity!(H, 0.3)      # singlet pairing
# H.sites = [nambu_s, spin_s, pos_qubits...] (aux_side=:pre)
# H.N is unchanged; the MPO now spans 2*2*N = 4N states

# Band structure tracing over Nambu and spin:
Akw = get_bands(H, omega_phys_vals)
# Band structure in the particle sector only:
Akw = get_bands(H, omega_phys_vals; aux_proj=[1, 1])
```

Band Structure with Sublattice Resolution

```
# Sublattice-aware honeycomb (H.sublattice_s is set)
H = get_Hamiltonian("honeycomb", (t=1.0,); L=8, Lx=4, Ly=4)
KPM_Tn(H, 200) # not needed -- get_bands runs online
Ak = get_bands(H, 200, 2, omega_vals) # sum over sublattices
Ak_A = get_bands(H, 200, 2, omega_vals;
                  sublat_proj=true, proj_sl=1) # sublattice A only

# High-symmetry path (G--M--K'--G for honeycomb)
res = get_bands(H, 200, 2, omega;
                kpath=[:G, :M, :Kp, :G], kpath_lattice=:honeycomb, num_x=40)
heatmap(1:size(res.Ak,2), omega, res.Ak; xticks=(res.ticks, res.labels))

# Legacy sublattice projection (models without H.sublattice_s, e.g. "hex_2d")
H2 = get_Hamiltonian("hex_2d", 1.0; L=8)
Ak = get_bands(H2, 200, 2, omega_vals; sublattice=true, proj_sl=1)
```

aux_side: postpending auxiliary DOF

By default (aux_side=:pre) auxiliary indices are prepended. To instead append them, pass position=:post to the first extension function:

```
H = get_Hamiltonian("square_2d", 1.0; L=8)
add_spin!(H; position=:post) # H.sites = [pos_qubits..., spin_s]
add_superconductivity!(H, 0.1) # H.sites = [pos_qubits..., spin_s, nambu_s]
# All subsequent add_* calls inherit H.aux_side = :post
```

Krylov Green's Function (krylov_tk.jl)

Computes the retarded single-particle Green's function $G(\omega) = (\omega + i\eta - H)^{-1}$ as an MPO by solving the vectorized linear system $[(\omega + i\eta - H) \otimes I] |G\rangle\rangle = |I\rangle\rangle$ via `ITensorMPS.linsolve` (a DMRG-like Krylov solver). No KPM Chebyshev expansion is required; the unscaled Hamiltonian is used directly.

Signature	Description
-----------	-------------

<code>get_green_krylov(H_mpo, sites, ω_phys; η=1e-2, nsweeps=12, maxdim=100, cutoff=1e-8, x0_mpo=nothing, ishermitian=false, tol=1e-10, maxiter=600, krylovdim=30, verbose=false) → MPO</code>	Low-level form. Solve on raw MPO <code>H_mpo</code> .
<code>get_green_krylov(H::TBHamiltonian, ω_phys; ...) → MPO</code>	High-level form. Forwards to the MPO overload using <code>H.mpo</code> and <code>H.sites</code> .

Key kwargs:

- `x0_mpo`: optional MPO warm-start (e.g. a cheap KPM Green's function). Vectorized via `_vec_mps_from_mpo` and passed as initial guess to `linsolve`.
- `ishermiteian`: set `true` only when the shifted operator is truly Hermitian (not physical for retarded GF with real $\eta > 0$).
- `krylovdim`: Krylov subspace dimension per site (default 30).

Usage:

```
G = get_green_krylov(H, omega; eta=0.05, nsweeps=20, maxdim=200)
dos = -imag(tr(G)) / pi
ldos = real(inner(psi_i, apply(G, psi_i))) # LDoS at site i
```

```
# Warm-start from KPM estimate
```

```
G_kpm = get_Green_retarded_from_Tn(H._tn_cache, H._tn_Ncheb, (omega-H.center)/H.scale; eta)
G      = get_green_krylov(H, omega; x0_mpo=G_kpm, nsweeps=8, maxdim=300)
```

Flake Masking (Flake_tk.jl)

Restricts a Hamiltonian to an irregular geometry by applying a smooth diagonal mask $M_i = \sigma(\text{sdf}(\mathbf{r}_i)/\sigma_0)$, where $\text{sdf}(\mathbf{r})$ is a signed-distance function (positive inside the flake) and σ is the logistic sigmoid. The resulting $H_{\text{flake}} = MHM$. Call `mask_hamiltonian` *before* `add_spin!`, `add_superconductivity!`, or any bilayer constructor.

SDF Primitives

Signature	Description
<code>sdf_interval(x_lo, x_hi) → x → Float64</code>	1D interval $[x_{\text{lo}}, x_{\text{hi}}]$; positive inside.
<code>sdf_disk(cx, cy, r) → (x,y) → Float64</code>	Disk of radius <code>r</code> centred at <code>(cx,cy)</code> ; positive inside.
<code>sdf_rect(cx, cy, w, h) → (x,y) → Float64</code>	Axis-aligned rectangle of full width <code>w</code> , height <code>h</code> ; positive inside.
<code>sdf_halfplane(nx, ny, d) → (x,y) → Float64</code>	Half-plane $n_x x + n_y y \geq d$; <code>(nx,ny)</code> need not be unit-length.
<code>sdf_annulus(cx, cy, r_in, r_out) → (x,y) → Float64</code>	Circular annulus; implemented as <code>sdf_subtract(disk_out, disk_in)</code> .
<code>sdf_convex_polygon(vertices) → (x,y) → Float64</code>	Convex polygon from CCW vertex list of <code>(x,y)</code> tuples.

CSG Combinators

Function	Description
<code>sdf_union(f, g)</code>	Boolean union: $\max(f, g)$; positive where either region holds.
<code>sdf_intersect(f, g)</code>	Boolean intersection: $\min(f, g)$; positive inside both.
<code>sdf_subtract(f, g)</code>	Subtraction: $\min(f, -g)$; removes g from f .

Mask Application

Signature	Description
<code>mask_hamiltonian(H, sdf; sigma=0.3, tol=1e-8, maxdim=200, cutoff=1e-8) → TBHamiltonian</code>	Apply smooth flake mask to H : learns M_i via QTCI, returns $H_{\text{flake}} = M H M$ as a new TBHamiltonian. sigma : boundary half-width in lattice units; smaller sigma = sharper edge.

Examples:

```
H = get_Hamiltonian("square_2d", 1.0; L=8, Lx=4, Ly=4)

# Disk
H_disk = mask_hamiltonian(H, sdf_disk(8.0, 8.0, 6.0); sigma=0.3)

# Triangular flake (intersection of three half-planes)
tri = sdf_intersect(sdf_intersect(
    sdf_halfplane(0, 1, 2.0),
    sdf_halfplane(-sqrt(3)/2, -0.5, -8.0)),
    sdf_halfplane(sqrt(3)/2, -0.5, -8.0))
H_tri = mask_hamiltonian(H, tri; sigma=0.2)

# Ring
H_ring = mask_hamiltonian(H, sdf_annulus(8.0, 8.0, 3.0, 7.0); sigma=0.3)
```

GPU Acceleration (GPU.tk.jl)

This file is the GPU companion to the CPU solvers in `src/solvers/` and `src/core/Utils.jl`. It makes large production runs (big L , many Chebyshev moments, fine spatial/ k grids) tractable by moving the dominant Chebyshev-recurrence and MPO-product cost onto a GPU via the NDTensors CUDA backend. Every public function mirrors a CPU counterpart with a `_gpu` suffix and (unless its own docstring says otherwise) accepts the same keyword arguments and returns the same shape of result.

Requirements:

```
using CUDA          # must be loaded *before* calling any *_gpu function
include("TensorBinding.jl"); using .TensorBinding
```

```
A = TensorBinding.get_ldos_spatial_gpu(H, 200, omega; maxdim=200)
```

GPU/CPU split (general pattern):

- **GPU:** Chebyshev recurrence (`KPM_Tn_gpu` / inline recursions), weighted MPO sums (`_weighted_mpo_sum_gpu`), MPO×MPO and MPO×MPS products and truncation, Hadamard products (`_hadamard_mpo_gpu`), aux-index projections (`_project_aux_gpu`), diagonal extraction (`extract_diagonal_to_mps_gpu`), and real-space/QFT sampling.
- **CPU:** one-time setup (Hamiltonian/operator construction, Tucker SVDs, k -/spatial-group bookkeeping, KPM weight matrices, McWeeny initial guesses) and the final per- ω scalar accumulation.

Precision (why F32): The NDTensors GPU backend requires `Float32` storage, so every MPO/MPS moved to GPU via `_to_gpu_mpo/_to_gpu_mps` is first cast to `ComplexF32`; results moved back via `_to_cpu_mpo/_to_cpu_mps` are promoted to `ComplexF64`. This is fine for the observables computed here, but `ComplexF32` eigendecomposition can produce NaN at very tight `cutoff` on large systems — affected functions warn (without altering the value) if `cutoff` is below a recommended floor, typically `1e-4` to `1e-6` depending on the routine. Per the project convention, these warnings never silently clamp `cutoff`; the user’s value is always passed through unchanged.

Real-space / bit-ordering conventions: Real-space sampling (`_eval_mps_bigendian_gpu`, `_eval_block_mps_gpu`, used by `get_ldos_spatial_gpu` and `get_scf_magnetization_gpu`) encodes the position index MSB-first across the site list, matching the CPU `eval_mps/binary_to_MPS` convention exactly. The QFT/bands path (`_eval_diag_mps_gpu`, used by `get_bands_gpu`) instead uses the legacy LSB-first convention required by the quantics-Fourier MPO. The two are **not** interchangeable — see `spatial_sampling_plan` in `Utils.jl` for how the shared sampler keeps them straight.

Production driver scripts that exercise this toolkit (magnetic-Hubbard SCF, magnetization/bands maps, spatial LDOS with the `:point/:block` samplers) live under `examples/nontracked/APSOS/`.

Spectral and Spatial Maps (DOS/LDOS/Bands)

Signature	Description
<code>get_bands_gpu(H::TBHamiltonian, Ncheb::Int, ω_{phys}_vals; kwargs...) → <code>Matrix{Float64}</code> or <code>NamedTuple(Ak, ticks, labels)</code></code>	GPU-accelerated $A(k, \omega)$ bands; mirrors the <code>TBHamiltonian</code> overload of <code>get_bands</code> . GPU handles the full Chebyshev MPO recurrence and the QFT sandwich applied to each Chebyshev moment; CPU handles k -group setup, the KPM weight matrix, and final scalar accumulation. All kwargs (<code>kpath</code> , <code>kpath_lattice</code> , <code>spin_proj</code> , <code>nambu_proj</code> , <code>layer_proj</code> , <code>sublat_proj</code> , <code>num_x</code> , <code>num_y</code> , <code>kernel</code> , <code>maxdim</code> , <code>cutoff</code> , ...) are identical to <code>get_bands</code> . Returns a plain <code>Matrix{Float64}</code> when no <code>kpath</code> is given, or <code>(Ak, ticks, labels)</code> for a high-symmetry path. If <code>Ak</code> comes back all-NaN, raise <code>cutoff</code> rather than lowering it.


```
get_ldos_spatial_gpu(H::TBHamiltonian,
Ncheb::Int,  $\omega$ _phys_vals;
x_groups=nothing, num_x=H.N,
num_y=nothing, num_avg=1, x_start=1,
x_end=H.N, grid=false, xwin=nothing,
ywin=nothing, box_half=0,
reduce=:point, sublattice=:auto,
kernel=:jackson, lambda=4.0,
maxdim=100, cutoff=1e-8,
⟨aux-projection kwargs⟩,
verbose=false, printinfo=false) →
Matrix{Float64} (shape  $N_\omega \times n_{\text{cols}}$ )
```

```
get_dos_stochastic_gpu(H::TBHamiltonian, GPU-accelerated stochastic DOS via MPS
Ncheb::Int,  $\omega$ _phys_vals; N_sample=50,
N_bound=0, seed=42, normalize=false,
dos_weighting=:trace,
kernel=:jackson, lambda=4.0, eta=0.0,
m_order=4, maxdim=100, cutoff=1e-8,
continuum_only=false, ⟨aux-projection
kwargs⟩, verbose=false,
printinfo=false) → Vector{Float64}
(length  $N_\omega$ )
```

GPU-accelerated version of `get_ldos_spatial` (MPO mode only; `:mps` mode is not available on GPU). `reduce=:point` (default) reads the LDOS at `num_x[ $\times$ num_y]` cells / `x_groups`, optionally box-averaged — coarse grids alias thin features. `reduce=:block` integrates over `num_x $\times$ num_y` power-of-two blocks by tracing out the within-block bits (gap-free, the scalable tool for large-scale edge-state maps); columns are row-major over coarse pixels, `col = ixp + iyp $\cdot$ num_x + 1`.

Column layout: no sublattice DOF $\rightarrow (N_\omega \times n_g)$, one column per pixel; sublattice resolved $\rightarrow (N_\omega \times n_g \times n_{\text{sub}})$, interleaved $[A_0, B_0, A_1, B_1, \dots]$; sublattice averaged (large scale / `:block`) $\rightarrow (N_\omega \times n_g)$. GPU handles the entire Chebyshev MPO recurrence, aux projections, diagonal extraction, and real-space scalar sampling; CPU handles the KPM weight matrix and output accumulation.

GPU-accelerated stochastic DOS via MPS Chebyshev KPM. For each random sample the scaled Hamiltonian MPO lives on GPU and the product-state MPS is transferred once before the recursion; the Chebyshev moments $\langle \psi_0 | T_n(\tilde{H}) | \psi_0 \rangle$ are scalars pulled to CPU at each step. `N_bound>0` (exciton bound-sector enrichment) and `dos_weighting=:sample` (unweighted sampled signal `avg_full + avg_bound`, useful for visualising exciton peaks hidden by continuum phase-space factors) are supported. For exciton Hamiltonians, `continuum_only=true` samples ordered electron-hole product states with $x_e \neq x_h$. `kernel=:hodc` selects the Higher-Order Delta Chebyshev reconstruction (`eta`, `m_order` control the contour); its weights already carry the full KPM normalisation, so no $\sqrt{1 - \omega^2}$ denominator is applied (`eta=0` falls back to $1/(N_{\text{cheb}} + 1)$). Otherwise `kernel` is a convolution kernel (`:jackson` default, `:lorentz` with `lambda`, `:fejer`, `:dirichlet`).

```
get_exciton_ldos_spatial_gpu(H::TBHamiltonian,
Ncheb::Int, ω_phys_vals;
X_list=nothing, X_groups=nothing,
x_groups=nothing, num_x=H.N,
num_avg=1, x_start=1, x_end=H.N,
kernel=:jackson, lambda=4.0, eta=0.0,
m_order=4, maxdim=100, cutoff=1e-8,
verbose=false, printinfo=false) →
Matrix{Float64} (shape  $N_\omega \times n_X$ )
```

GPU-accelerated spatial exciton LDOS. For each bound exciton position X (electron = hole = X , 1-indexed in $1:H.N$), the local spectral weight $A(X, \omega) = \langle X, X | \delta(\omega - H) | X, X \rangle$ is reconstructed from the Chebyshev moments $\mu_n = \langle X, X | T_n(\tilde{H}) | X, X \rangle$, with one GPU MPS Chebyshev recursion run per X starting from $|X, X\rangle = \text{mpsexciton}(X, H.\text{sites})$; moments are scalars pulled to CPU. Spatial / batched GPU analog of the CPU `get_exciton_ldos` (same KPM-weight-matrix reconstruction). `X_list` selects positions directly; `X_groups` / `x_groups` select explicit position groups and average all probes in each group into one output column; otherwise `num_x` coarse groups are built over `x_start:x_end` with the same stride convention as `get_ldos_spatial`. `kernel=:hodc` behaves as in `get_dos_stochastic_gpu`.

Topology

Signature	Description
<pre>get_C_gpu(H::TBHamiltonian, xfunc=nothing, yfunc=nothing; method=:mcweeny, fermi=0.0, l=nothing, Λ=10, Lambda=nothing, Nchebychev=300, maxdim=500, cutoff=1e-8, Nel=nothing, quenched=true, printinfo=false) → Function</pre>	GPU-accelerated real-space Chern marker. Mirrors <code>get_C</code> exactly but runs all MPO×MPO products (projector assembly and C_1–C_4 construction) on GPU in F32, and returns the same closure <code>C_at(uc::Int) → ComplexF64</code> as <code>get_C</code>. Differences from <code>get_C</code>: all <code>apply/truncate!</code> run on GPU tensors (F32); a warning is emitted if <code>cutoff < 1e-6</code> since <code>ComplexF32</code> eigendecompositions can NaN at tight cutoffs on large systems; the projector is built on CPU via <code>purification_initial_guess</code> (<code>method=:KPM</code> via <code>_get_projector</code>) and then moved to GPU — for <code>method=:mcweeny/sp2</code> the purification loop itself runs on GPU; <code>sequential</code> mode is not supported (non-sequential <code>quenched</code> is always used). All other keyword arguments are identical to <code>get_C</code>.

SCF (Magnetic Hubbard)

Signature	Description
-----------	-------------

```
scf_magnetic_hubbard_gpu(H0::TBHamiltonian,
U::Union{Number,MPO};
initial_up=nothing,
initial_dn=nothing, background=0.5,
Nel_up=H0.N÷2, Nel_dn=H0.N÷2,
fermi=0.0, scale=H0.scale,
purification_scale_padding=1.05,
max_scf_iter=30, scf_tol=1e-6,
mix=0.4, maxdim=100, cutoff=1e-8,
purif_maxiter=40, purif_tol=1e-6,
verbose=true) → NamedTuple
```

```
get_scf_magnetization_gpu(res;
num_x=0, num_y=nothing, num_avg=1,
x_start=1, x_end=N, x_groups=nothing,
box_half=0, Lx=nothing) → (values,
centers, groups, n_up, n_dn)
```

```
get_scf_bands_gpu(res, Ncheb::Int,
omega; kwargs...) → (Ak, omega,
ticks, labels)
```

GPU-accelerated two-channel collinear magnetic mean-field loop for the on-site Hubbard model:

$$H_{\uparrow} = H_{0\uparrow} + U \cdot \text{diag}(n_{\downarrow} - \text{background}),$$

$$H_{\downarrow} = H_{0\downarrow} + U \cdot \text{diag}(n_{\uparrow} - \text{background}).$$

The SCF iteration keeps density profiles, Hartree MPOs, Hamiltonian MPOs, density matrices, RMS checks, and mixing on GPU; CPU objects are built only for initialization and for the returned compatibility fields. Only `density_method=:mcweeny` is supported (grand-canonical at `fermi`); for particle-number-fixed SP2 use the CPU `scf_magnetic_hubbard`. A concrete nonzero `scale` is **required** so the GPU initial guess can be formed without estimating spectral bounds on CPU during the loop. A warning (not a clamp) is emitted if `cutoff < 1e-5`. Returns (`converged`, `iterations`, `rms_error`, `rho_up`, `rho_dn`, `density_up_mpo`, `density_dn_mpo`, `H_up`, `H_dn`, `rho_up_gpu`, `rho_dn_gpu`, `density_up_mpo_gpu`, `density_dn_mpo_gpu`, `H_up_mpo_gpu`, `H_dn_mpo_gpu`, `history`) — the `_gpu` fields retain GPU-resident objects for reuse by `get_scf_magnetization_gpu/get_scf_bands_gpu`. Sample the converged magnetic SCF density matrices on GPU and extract only the final scalar values. If `res` carries GPU density MPOs from `scf_magnetic_hubbard_gpu` (`density_up/dn_mpo_gpu`), they are reused directly; otherwise the CPU density MPOs are uploaded once. Sampling groups come from the shared `spatial_sampling_plan` (via the internal `_tb_spatial_groups_gpu` wrapper), and each sampled point is evaluated in the same big-endian real-space convention as `binary_to_MPS`. `values = (n↑ - n↓)/2` is the local magnetization $\langle S_z \rangle(r)$. Compute spin-summed mean-field bands from a converged magnetic SCF result. Deliberately separate from `scf_magnetic_hubbard_gpu`: it initializes from the CPU `res.H_up/res.H_dn`, and each `get_bands_gpu` call uploads once and keeps the Chebyshev/QFT accumulation on GPU, extracting only scalars. `Ak = Ak_up + Ak_dn`; `kwargs` are forwarded verbatim to both `get_bands_gpu` calls.

Experimental: Conductivity (Not Working Yet)

The module header for `GPU.tk.jl` explicitly lists the following two entry points under “Conductivity (experimental/not working yet)”. They are documented here for completeness only — their signatures and intended GPU/CPU split match `get_conductivity_ward/get_conductivity_cheb2d` from `Conductivity.tk.jl`, but they should not be relied upon for production results until this note is updated.

Signature	Description
<pre>get_conductivity_ward_gpu(H1::TBHamiltonian, H2::TBHamiltonian, ωlist; q_int=1, Lx=H1.L÷2, q_sq=nothing, Ncheb=50, maxdim=200, cutoff=1e-8, ε_F=0.0, P_method=:purification, purify_method=:mcweeny, purify_maxdim=40, purify_maxiters=30, purify_tol=1e-5, η=1e-3, coeff_tol=1e-12, kernel=:jackson, tucker_tol=1e-3, tucker_maxrank=20, hooi_iters=3, qft_tol=1e-9, qft_maxdim=100, verbose=false) → Vector{ComplexF64}</pre>	Experimental / not working yet. GPU-accelerated Ward-identity $\sigma(\omega)$. GPU handles the Chebyshev recurrence, density-matrix transfer, weighted MPO sums, MPO×MPO multiplications, and Hadamard products; CPU handles <code>replace_sites</code>, the QFT sandwich, inner products, and per-ω accumulation. Kwargs otherwise mirror <code>get_conductivity_ward</code> (threading is removed — use <code>julia -t N</code> at the process level instead).
<pre>get_conductivity_cheb2d_gpu(H1::TBHamiltonian, H2::TBHamiltonian, ωlist; mu_a=:x, mu_b=:x, Ncheb=50, maxdim=200, cutoff=1e-8, ε_F=0.0, P_method=:purification, purify_method=:mcweeny, purify_maxdim=40, purify_maxiters=30, purify_tol=1e-5, η=1e-3, kernel=:jackson, tucker_tol=1e-3, tucker_maxrank=20, hooi_iters=3, tol_dr=1e-8, verbose=false) → Vector{Float64}</pre>	Experimental / not working yet. GPU-accelerated 2D-Chebyshev $\sigma(\omega)$, with the same GPU/CPU split as <code>get_conductivity_ward_gpu</code>.

Internal helpers (not part of the public API): `KPM.Tn_gpu` runs the GPU Chebyshev recurrence and returns the moment MPOs (optionally restricted to `keep_indices`); `extract_diagonal_to_mps_gpu` and `density_profile_from_dm_gpu` are GPU-resident analogs of `extract_diagonal_to_mps` and `density_profile_from_dm` (`mode=:complement` returns $1 - \text{diag}(D)$ on GPU); `_to_gpu_mpo/_to_gpu_mps/_to_cpu_mpo/_to_cpu_mps` handle the F32 round-trip conversions; `_hadamard_mpo_gpu`, `_apply_qft_conj_gpu`, `_project_aux_gpu`, and `_weighted_mpo_sum_gpu` are the GPU primitives used throughout the spectral/spatial map and topology routines; `_check_gpu` verifies CUDA is loaded and `_gpu_gc!` runs incremental GPU garbage collection between steps to bound memory growth in long recursions.

T-Junction Geometries (TJunction.tk.jl)

Builds T/Y-junction Hamiltonians via a spin-1 (dim-3) chain-label index: a T-junction (Y-junction) connects three identical 1D chains at a common site. The dim-3 “**TJunction**” index postpended to the MPO labels the branch ($1 \equiv | - 1 \rangle$, $2 \equiv | 0 \rangle$, $3 \equiv | + 1 \rangle$); all three branches

share the same intra-chain Hamiltonian, and a 3×3 operator acting at a designated junction site couples them.

Atom index convention for `H.geometry` (matches kagome — branch-fast): for combined 1-indexed atom i , $n = (i - 1) \div 3$ is the 0-indexed chain position and $s = (i - 1) \bmod 3 + 1$ is the branch ($1 = |-1\rangle$, $2 = |0\rangle$, $3 = |+1\rangle$); total atoms $3N$ with $N = 2^L$.

TJunction Index and Geometry

Signature	Description
<code>tjunction_index() → Index</code>	Dim-3 Index tagged "TJunction". States: 1 = branch $ -1\rangle$, 2 = branch $ 0\rangle$, 3 = branch $ +1\rangle$.
<code>tjunction_positions(N::Int, junction_site::Int) → Matrix{Float64}</code>	$(3N \times 2)$ real-space position matrix for a Y-junction of three chains of length N, meeting at <code>junction_site</code> (0-indexed). Branches radiate symmetrically from the origin at 0, 120, 240; position along a branch is the signed distance $n - \text{junction_site}$ (positive = away from junction).

Hamiltonian Construction

Signature	Description
<code>add_tjunction!(H::TBHamiltonian, t_j; junction_site=0, coupling=nothing, cutoff=1e-8, maxdim=200) → TBHamiltonian</code>	Extend a 1D TBHamiltonian in place to a T/Y-junction by postpending a dim-3 "TJunction" index: $H_{\text{TJ}} = I_3 \otimes H_{\text{chain}} + J(t_j) \otimes \text{junction_site}\rangle\langle\text{junction_site} $. The default 3×3 coupling is the symmetric Y-junction $J = t_j \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$; pass a custom 3×3 matrix via <code>coupling</code> (e.g. for a linear T-junction where only two of the three branches are mutually coupled). Requires H to have no existing auxiliary indices (spin, Nambu, layer, sublattice). Sets <code>H.geometry</code> to the 2D Y-junction layout (<code>tjunction_positions</code>), <code>H.sublattice_s</code> to the TJunction index, and <code>H.aux_side=:post</code> ; invalidates all caches.
<code>tjunction_hamiltonian(L::Int, t::Number, t_j::Number; junction_site=0, coupling=nothing, boundary=:open, cutoff=1e-8, maxdim=200) → TBHamiltonian</code>	Standalone constructor: three identical 1D nearest-neighbour chains (hopping <code>t</code> , 2^L sites per branch, <code>boundary</code> BC) meeting at <code>junction_site</code> with coupling <code>t_j</code> . Built via <code>get_Hamiltonian("chain_1d", t; L, boundary)</code> followed by <code>add_tjunction!</code> . Sets a rough spectral scale $2.5 t + 2 \max(J)$.

<pre> tjunction_lattice_hamiltonian(Lx::Int, Ly::Int, L::Int, t::Number, t_j::Number, t_inter::Number; junction_site=0, coupling=nothing, boundary=:open, cutoff=1e-8, maxdim=200) → TBHamiltonian </pre>	Triangular lattice of T/Y-junctions with $2^{L_x} \times 2^{L_y}$ unit cells, each a T-junction with three arms of 2^L sites. Site order is [iy bits(L_y), ix bits(L_x), chain(L), tj_s] (each block MSB-first). Structure: (1) intra-arm NN hopping t along each chain; (2) junction coupling $t_j \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$ (or <code>coupling</code>) at <code>junction_site</code>, acting on the branch index; (3) inter-cell hopping <code>t_inter</code> between arm ends across unit cells: an x-shift (-1) couples arm 1 (330) $\leftrightarrow$ arm 3 (210), a y-shift ($+N_x$) couples arm 3 $\leftrightarrow$ arm 2 (90), and a diagonal shift ($+N_x-1$) couples arm 2 $\leftrightarrow$ arm 1. <code>H.sublattice_s</code> stores the dim-3 TJunction index (<code>H.aux_side=:post</code>); <code>H.L=Lx+Ly+L</code>, <code>H.N=2^(Lx+Ly)·2^L</code>. Spectral scale $2.5 t + 2 \max(J) + 2 t_{\text{inter}}$.
---	---

Usage:

```

# Symmetric Y-junction, junction at left end (site 0)
H = tjunction_hamiltonian(8, 1.0, 0.5)

# Linear T-junction (branches 1<->2 and 2<->3 only)
H3 = tjunction_hamiltonian(8, 1.0, 0.5; coupling=[0 1 0; 1 0 1; 0 1 0])

# 4x4 triangular lattice of T-junctions, arms of length 8
Hlat = tjunction_lattice_hamiltonian(2, 2, 3, 1.0, 0.5, 0.8)

```

The internal helper `_site_projector_mpo(L, pos_sites, n)` builds the exact rank-1 (bond-dim-1) projector $|n\rangle\langle n|$ for 0-indexed site n on L position qubits, via `OpSum` with "`sigma_u`"/"`sigma_d`" factors — no QTCI required.

Non-Hermitian Extensions (NH_tk.jl)

`NH_tk.jl` adds non-Hermitian (NH) Hamiltonian support on top of `TBHamiltonian`. The core construction hermitizes a non-Hermitian single-particle MPO by adding one dim-2 auxiliary block index:

$$H_{\text{NH}}(z) = \begin{pmatrix} 0 & zI - H \\ (zI - H)^\dagger & 0 \end{pmatrix} = |1\rangle\langle 2| \otimes (zI - H) + |2\rangle\langle 1| \otimes (zI - H)^\dagger,$$

using the same `prepend_op` machinery used for layers and BdG/Nambu blocks. The resulting Hermitian `TBHamiltonian` can be fed to existing MPO/KPM routines. A second family of helpers builds non-Hermitian terms directly on the original Hamiltonian (complex onsite potentials, loss/gain, non-reciprocal “skin-effect” hopping) without hermitization, and a final family implements the KPM-based non-Hermitian spectral-function reconstruction on a complex energy grid.

Hermitization

Signature	Description
<code>nh_block_index() → Index</code>	Dim-2 auxiliary block <code>Index</code> tagged "Qubit,NHBlock". State 1 is the upper block, state 2 the lower block; the "Qubit" tag keeps generic <code>MPO(sites, "Id")</code> construction compatible with existing KPM code paths.
<code>hermitized_hamiltonian(H::TBHamiltonian, z=0, block_s=nh_block_index(), cutoff=1e-8, maxdim=200, scale=0.0, convention=:z_minus_H) → TBHamiltonian</code>	Build the Hermitian block Hamiltonian $\begin{pmatrix} 0 & zI-H \\ (zI-H)^\dagger & 0 \end{pmatrix}$ as a <code>TBHamiltonian</code> on <code>[block_s; H.sites...]</code> . <code>convention=:z_minus_H</code> (default) uses $zI - H$; <code>convention=:H_minus_z</code> uses $H - zI$. <code>scale=0.0</code> keeps the usual lazy KPM spectral-bound estimation.
<code>hermitize(H::TBHamiltonian; z=0, cutoff=1e-8, maxdim=200, scale=0.0, convention=:z_minus_H) → NonHermitianHamiltonian</code>	Build a <code>NonHermitianHamiltonian</code> wrapper without modifying <code>H</code> (first form), or rebuild the hermitized block Hamiltonian from <code>NH.parent</code> , optionally at a new reference point <code>z</code> (second form).
<code>hermitize(NH::NonHermitianHamiltonian; z=NH.z, cutoff=1e-8, maxdim=200, scale=0.0, convention=:z_minus_H) → NonHermitianHamiltonian</code>	<code>NonHermitianHamiltonian</code> has fields <code>parent</code> (original <code>TBHamiltonian</code> , not modified in place), <code>z</code> (complex reference point in $zI - H$), <code>block_s</code> (the dim-2 "NHBlock" index), and <code>hermitized</code> (the Hermitian <code>TBHamiltonian</code>). Avoid calling tight-binding mutation helpers (e.g. <code>add_onsite!</code>) on <code>hermitized</code> ; mutate <code>parent</code> and call <code>hermitize</code> again.

Non-Hermitian Model Building

These helpers add non-Hermitian terms directly to `H.mpo` on a position-only Hamiltonian (no spin/Nambu/layer/sublattice auxiliaries); calling them on a Hamiltonian with existing auxiliary indices raises an error.

Signature	Description
<code>add_nh_onsite!(H::TBHamiltonian, v; Lx=nothing, tol=1e-8, maxdim=200, type=ComplexF64) → TBHamiltonian</code>	Non-Hermitian counterpart of <code>add_onsite!</code> : adds a possibly complex onsite potential <code>v</code> , preserving complex values by default. <code>v</code> may be a <code>Number</code> (e.g. <code>1im*gamma</code>), <code>v(n)</code> with $n = 0, \dots, H.N - 1$, or <code>v(ix,iy)</code> (0-indexed, requires <code>Lx</code>).
<code>loss_profile_mpo(H::TBHamiltonian, f; Lx=nothing, type=Float64, space=:full) → MPO</code>	Build the real diagonal profile MPO <code>diag(f)</code> used for loss/gain terms (no <code>im</code> factor, no hermitization). <code>space=:full</code> (default) builds on the full <code>H.sites</code> ($n = 0, \dots, 2^{ H.sites } - 1$); <code>space=:position</code> builds on position qubits only (before auxiliary spaces are attached).

<code>nh_imag_onsite_mpo(H::TBHamiltonian, f; prefactor=1im, Lx=nothing, type=Float64, space=:full) → MPO</code>	prefactor · diag(f), built via <code>loss_profile_mpo</code> then scaled by the (generally imaginary) <code>prefactor</code> . Returns a term on the original Hilbert space, not an NH hermitized block.
<code>add_loss!(H::TBHamiltonian, f; coefficient=-1im, space=:full, Lx=nothing, tol=1e-8, maxdim=200, type=Float64) → TBHamiltonian</code>	Add coefficient · diag(f) to <code>H.mpo</code> (only modifies <code>H.mpo</code> ; does not build a hermitized NH block).
<code>add_nh_imag_onsite!(H::TBHamiltonian, f; prefactor=1im, kwargs...) → TBHamiltonian</code>	Backward-compatible wrappers around <code>add_loss!</code> . <code>add_nh_imag_onsite!</code> forwards <code>prefactor</code> as <code>coefficient</code> ; <code>add_nh_loss!</code> defaults to <code>prefactor=-1im</code> (use <code>prefactor=1im</code> for gain or the convention $if(x)$).
<code>add_nh_loss!(H::TBHamiltonian, f; prefactor=-1im, kwargs...) → TBHamiltonian</code>	
<code>nh_nonreciprocal_hopping_mpo(H::TBHamiltonian, t_forward, t_backward; nn=1, tol=1e-8, maxdim=200, type=ComplexF64) → MPO</code>	Position-space MPO $t_{\text{forward}} K_+^{nn} + t_{\text{backward}} K_-^{nn}$ (no Hermiticity imposed), where $K_{\pm}$ are forward/backward <code>shift_mpo</code> operators. <code>t_forward/t_backward</code> may be Numbers or one-argument functions <code>t(n)</code> (0-indexed).
<code>add_nh_nonreciprocal_hopping!(H::TBHamiltonian, t_forward, t_backward; nn=1, tol=1e-8, maxdim=200, type=ComplexF64) → TBHamiltonian</code>	<code>add_nh_nonreciprocal_hopping_mpo</code> to <code>H.mpo</code> in place — the skin-effect helper.
<code>add_nh_skin_hopping!(H::TBHamiltonian, t, g; nn=1, convention=:exp, tol=1e-8, maxdim=200) → TBHamiltonian</code>	Convenience wrapper for non-reciprocal skin hopping. <code>convention=:exp</code> (default) uses $t_R = te^g$, $t_L = te^{-g}$; <code>convention=:linear</code> uses $t_R = t + g$, $t_L = t - g$.

Non-Hermitian KPM Spectral Function

The non-Hermitian KPM path computes an auxiliary “partial” Chebyshev recursion on the hermitized block Hamiltonian and reconstructs a scalar spectral weight $\text{dos}(z)$ (and, optionally, the site-resolved LDOS) at a complex reference point z . With $A = H_{\text{hermitized}}/\text{scale}$ and block source $S = |\text{row}\rangle\langle\text{col}| \otimes I$, the recurrence is

$$P_0 = 0, \quad P_1 = S, \quad P_k = 2ST_{k-1}(A) + 2AP_{k-1} - P_{k-2},$$

run in parallel with the ordinary Chebyshev $T_k(A)$.

Signature	Description
<code>nh_block_source(NH::NonHermitianHamiltonian, row=2, col=1) → MPO</code>	Off-diagonal block source $ \text{row}\rangle\langle\text{col} \otimes I$. The default <code>row=2</code> , <code>col=1</code> matches the legacy <code>I_ldn</code> source used in the non-Hermitian KPM recursion.
<code>nh_block_source(Hh::TBHamiltonian, block_s::Index; row=2, col=1) → MPO</code>	

<code>nh_kpm_partials(NH::NonHermitianHamiltonian, n::Int; source=nothing, source_row=2, source_col=1, scale=nothing, maxdim=100, cutoff=1e-8) → Vector{MPO}</code>	Compute the partial-recursion MPOs $P_0, \dots, P_{2n-1}$ (length $2n$) described above. Requires a nonzero <code>scale</code> (defaults to <code>Hh.scale/NH.hermitized.scale</code>).
<code>nh_kpm_partials(Hh::TBHamiltonian, n::Int; source::MPO, scale=nothing, maxdim=100, cutoff=1e-8) → Vector{MPO}</code>	
<code>contract_nh_block(W::MPO, block_s::Index; row=2, col=1) → MPO</code>	Extract the <code>(row,col)</code> block of an MPO whose first site is <code>block_s</code> , returning an MPO on the remaining sites.
<code>nh_reconstruct_spectral_mps(partials, n::Int, block_s::Index; maxdim=100, row=2, col=1) → (A_mps, dos)</code>	Jackson reconstruction of the even partial terms, $A = \frac{2}{\pi^2(2n+1)} \sum_{l=2,4,\dots,2n} (-1)^{l/2-1} g_l \text{diag}(P_l)$, applied after extracting the <code>(row,col)</code> NH block and its diagonal as an MPS for each partial (<code>nh_preprocess_partials</code>). Returns the diagonal spectral MPS <code>A_mps</code> and its summed value <code>dos</code> .
<code>nh_reconstruct_spectral_mpo(partials, n::Int, NH::NonHermitianHamiltonian; maxdim=400, cutoff=1e-8, rotate_row=1, rotate_col=2, diag_block=1) → (ldos_mps, dos, rotated_mpo)</code>	High-bond-dimension alternative: reconstruct the full partial MPO on the hermitized block space, left-multiply by $ \text{rotate_row}\rangle\langle\text{rotate_col} \otimes I$ (<code>nh_block_source</code>), then take its trace (<code>dos</code>) and the diagonal of its <code>(diag_block,diag_block)</code> block (<code>ldos_mps</code>). Can represent all sites at once but needs a much larger <code>maxdim</code> than <code>nh_reconstruct_spectral_mps</code> .
<code>nh_spectral_function(NH::NonHermitianHamiltonian, n::Int; scale=nothing, maxdim=100, cutoff=1e-8, source_row=2, source_col=1, block_row=2, block_col=1) → (A_mps, dos, partials)</code>	Convenience wrapper: <code>nh_kpm_partials</code> followed by <code>nh_reconstruct_spectral_mps</code> at the reference point stored in <code>NH.z</code> .
<code>nh_spectral_function_allsite_mpo(NH::NonHermitianHamiltonian, n::Int; scale=nothing, maxdim=400, cutoff=1e-8, source_row=2, source_col=1, rotate_row=1, rotate_col=2, diag_block=1) → (ldos_mps, dos, rotated_mpo, partials)</code>	Non-iterative, NH KPM path: <code>nh_kpm_partials</code> followed by <code>nh_reconstruct_spectral_mpo</code> .
<code>nh_spectrum_grid(H::TBHamiltonian, xlims, nx::Int, ylims, ny::Int, n::Int; scale::Real, convention=:z_minus_H, maxdim=100, cutoff=1e-8) → (xgrid, ygrid, Z)</code>	Evaluate the non-Hermitian KPM scalar spectral weight <code>dos(z)</code> on a rectangular complex-energy grid $z = x + iy$, $x \in \text{xlims}$ (n_x points), $y \in \text{ylims}$ (n_y points). For each grid point, builds <code>hermitize(H; z, scale, maxdim, cutoff, convention)</code> and calls <code>nh_spectral_function</code> ; <code>Z[iy,ix] = dos</code> .

Internal helpers: `nh_ones_mps(sites)` builds the bond-dim-1 MPS with every amplitude = 1

(used to sum `A_mps` into `dos`); `nh_jackson_weights(N)` returns the Jackson kernel weights for N moments; `nh_position_sites_only`, `nh_diagonal_mpo`, `nh_profile_diagonal_mpo`, `nh_fullspace_diagonal_mpo` and `nh_directional_hop` are shared dispatch/validation helpers for the model-building functions above.

Quasi-Particle Interference (QPI_tk.jl)

Computes the Quasiparticle Interference (QPI) pattern produced by a single on-site impurity via the standard T -matrix-free difference method: KPM LDOS of the clean and perturbed Hamiltonians are subtracted in real space, then Fourier transformed.

Pipeline:

- Build the impurity potential $V \cdot |x_0\rangle\langle x_0|$ analytically via `OpSum` (bond dimension 1).
- Run the KPM Chebyshev expansion for the clean (`H`) and perturbed (`H_imp`) systems and extract the site-resolved diagonal LDOS as `Vector{MPS}` (one MPS per ω), online — only 3 MPOs per system live at a time, no full T_n cache is stored.
- Subtract: $\delta A_{\text{mps}}(\omega) = A_{\text{imp}}(\omega) - A_{\text{clean}}(\omega)$, then remove the spatial mean ($\text{tr}[\delta A]/N$) so the $q = 0$ component vanishes.
- Optionally apply a smooth disk/interval apodization window (same SDF machinery as `mask_hamiltonian`) to suppress edge-induced ringing.
- Apply the forward QFT MPO directly to δA_{mps} (no `conjugate_by_qft` needed — the diagonal is already an MPS) to get $\delta \tilde{A}_{\text{mps}}$, then read out $\text{QPI}(k, \omega) = |\langle k | \delta \tilde{A}_{\text{mps}} \rangle|^2$ using LSB-first bit encoding (site 1 = bit 0), consistent with the `TCI.reverse` QFT convention.

Signature	Description
-----------	-------------

<pre>get_qpi(H::TBHamiltonian, Ncheb::Int, ω_phys_vals; impurity_site=nothing, V=1.0, impurity_mode=:delta, sigma=1.5, window_fraction=nothing, window_sigma=1.5, kernel=:jackson, lambda=4.0, maxdim=100, cutoff=1e-8, verbose=false) → Matrix{Float64} (shape $N_ω \times H.N$)</pre>	Compute the QPI pattern $QPI(k, \omega)$. H must carry no auxiliary DOFs (spin, Nambu, layer, sublattice); call <code>get_qpi</code> before <code>add_spin!</code> etc. <code>impurity_site</code> (1-indexed, default <code>central_index(H)</code>) places the impurity. <code>impurity_mode=:delta</code> (default) builds the exact rank-1 projector $V x_0\rangle\langle x_0$ via <code>OpSum</code>; <code>impurity_mode=:gaussian</code> instead adds a smooth potential $V \exp(- \mathbf{r} - \mathbf{r}_0 ^2/2\sigma^2)$ via <code>add_onsite!</code> (lower KPM bond dimension; requires <code>H.geometry</code>, with <code>sigma</code> in physical distance units). <code>window_fraction</code>, if set, builds a circular (2D, via <code>sdf_disk</code>) or interval (1D, via <code>sdf_interval</code>) apodization mask of radius <code>window_fraction</code> $\times$ (min half-extent of the bounding box), with sigmoid roll-off half-width <code>window_sigma</code> lattice units (requires <code>H.geometry</code>; skipped with a warning if absent). <code>kernel/lambda</code> select the KPM kernel (<code>:jackson</code> or <code>:lorentz</code>). Returns a $(N_ω \times H.N)$ matrix where column $k+1$ (1-based) corresponds to 0-indexed momentum k.
--	---

Usage:

```
H      = get_Hamiltonian("square_2d", 1.0; L=8, Lx=4)
omega  = [0.0]
qpi    = get_qpi(H, 80, omega; V=2.0, window_fraction=0.8)
# Reshape to 2D: reshape(qpi[1,:], 2^Lx, 2^Ly)'
```

The internal helper `_impurity_mpo(x0, L, sites, V)` builds the on-site impurity potential $V \cdot |x_0 - 1\rangle\langle x_0 - 1|$ as a bond-dimension-1 MPO via `OpSum`, using `"projUp"/"projDn"` operators and the same big-endian (MSB-first) quantics encoding as `binary_to_MPS`.